

Езикови процесори. Въведение

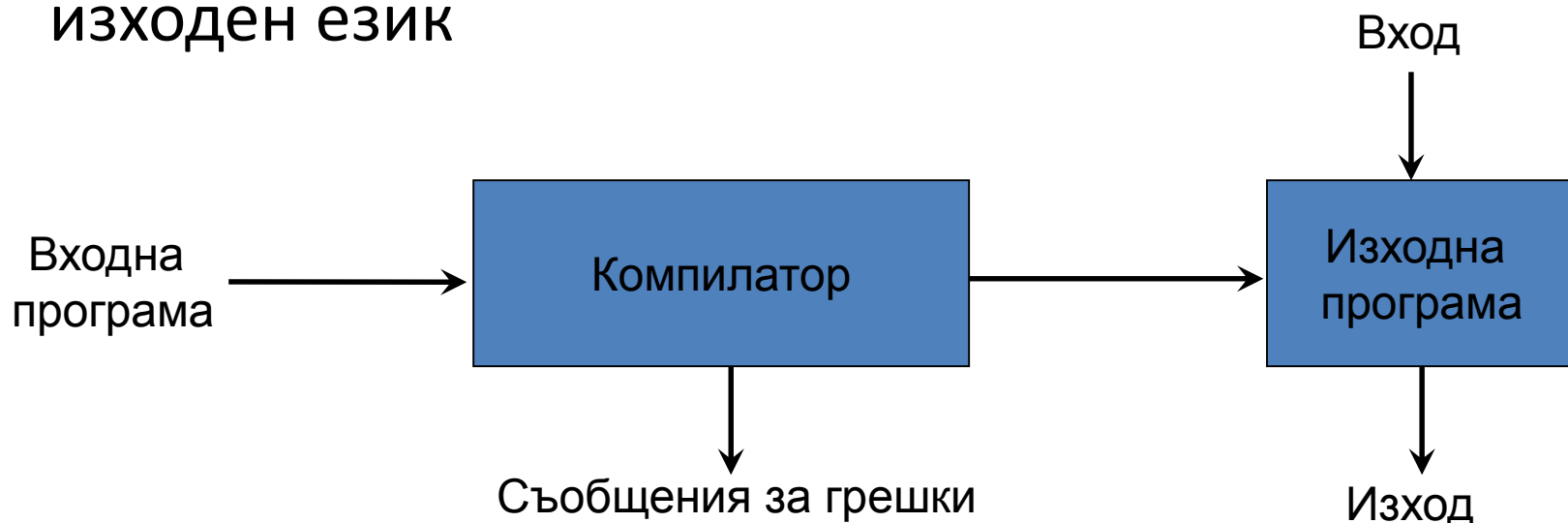
Езиков процесор

- Езиковият процесор е програма, която чете програма, написана на един език (*входен език*) и я превежда (транслира) в еквивалентна програма на друг език (*изходен език*).
- Важна роля на езиковия процесор е да регистрира и извежда съобщения за грешки, открити във входната програма по време на транслацията.

Компилатори и интерпретатори

- *“Компилиране”*

- Транслиране на програма, написана на входен език, в семантично еквивалентна програма на изходен език

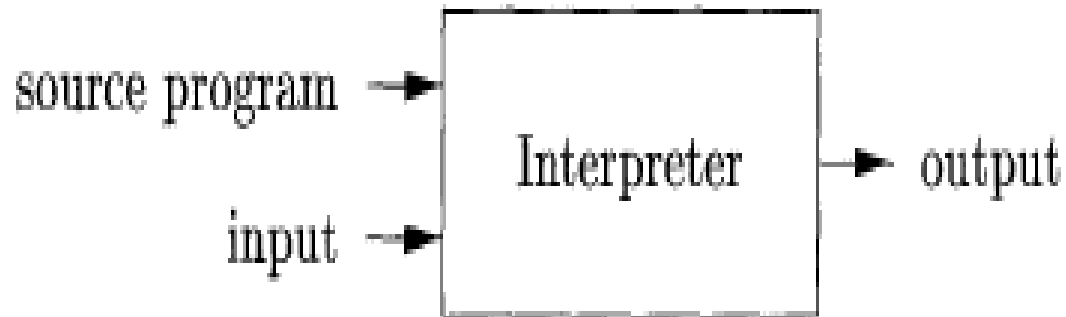


Ако изходната програма е изпълнима програма на машинен език, тя може да бъде стартирана от потребител за обработване на входни данни и генериране на резултати



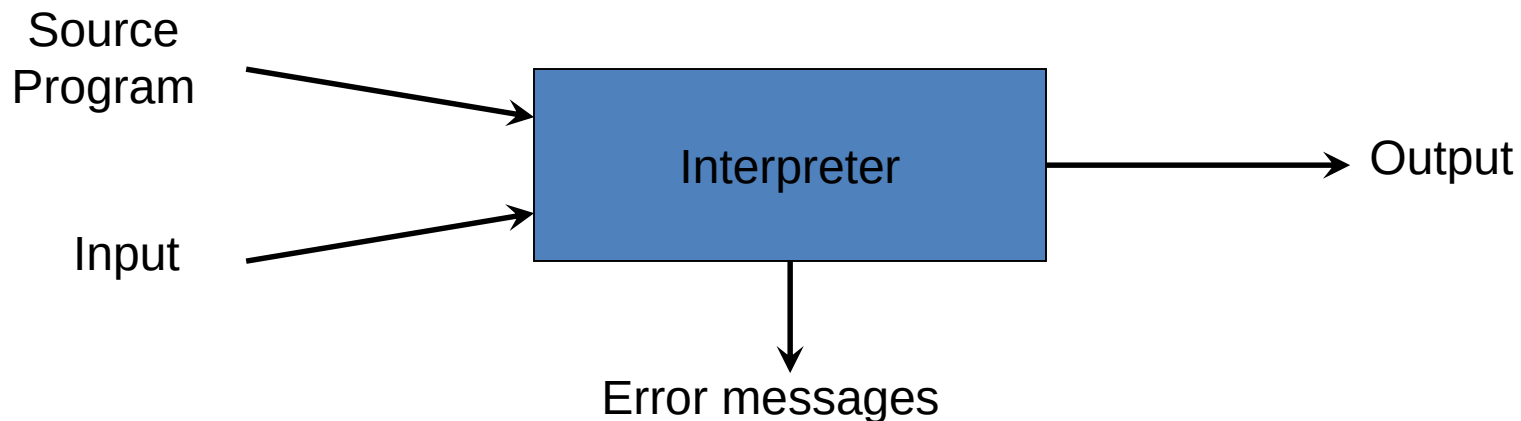
Интерпретатор

Интерпретатор е друг вид езиков процесор. Вместо да генерира изходна програма интерпретаторът директно изпълнява операциите, зададени във входната програма с въведени входни данни.



Компилатори и интерпретатори (продължение)

- *“Интерпретиране”*
 - Изпълнение на операциите от входната програма

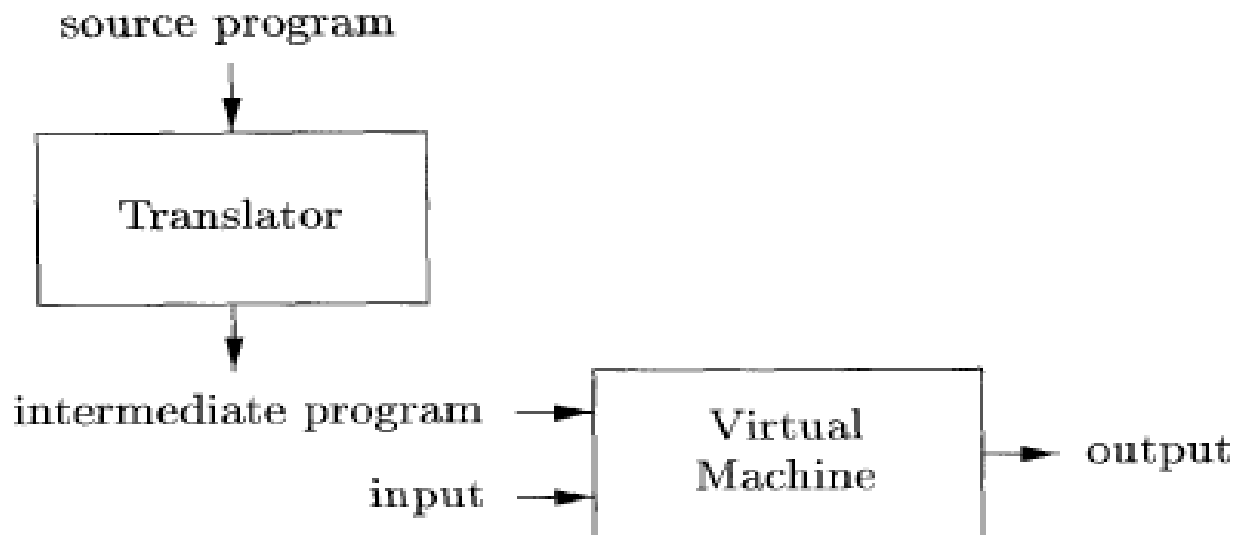


Компилатори и интерпретатори (сравнение)

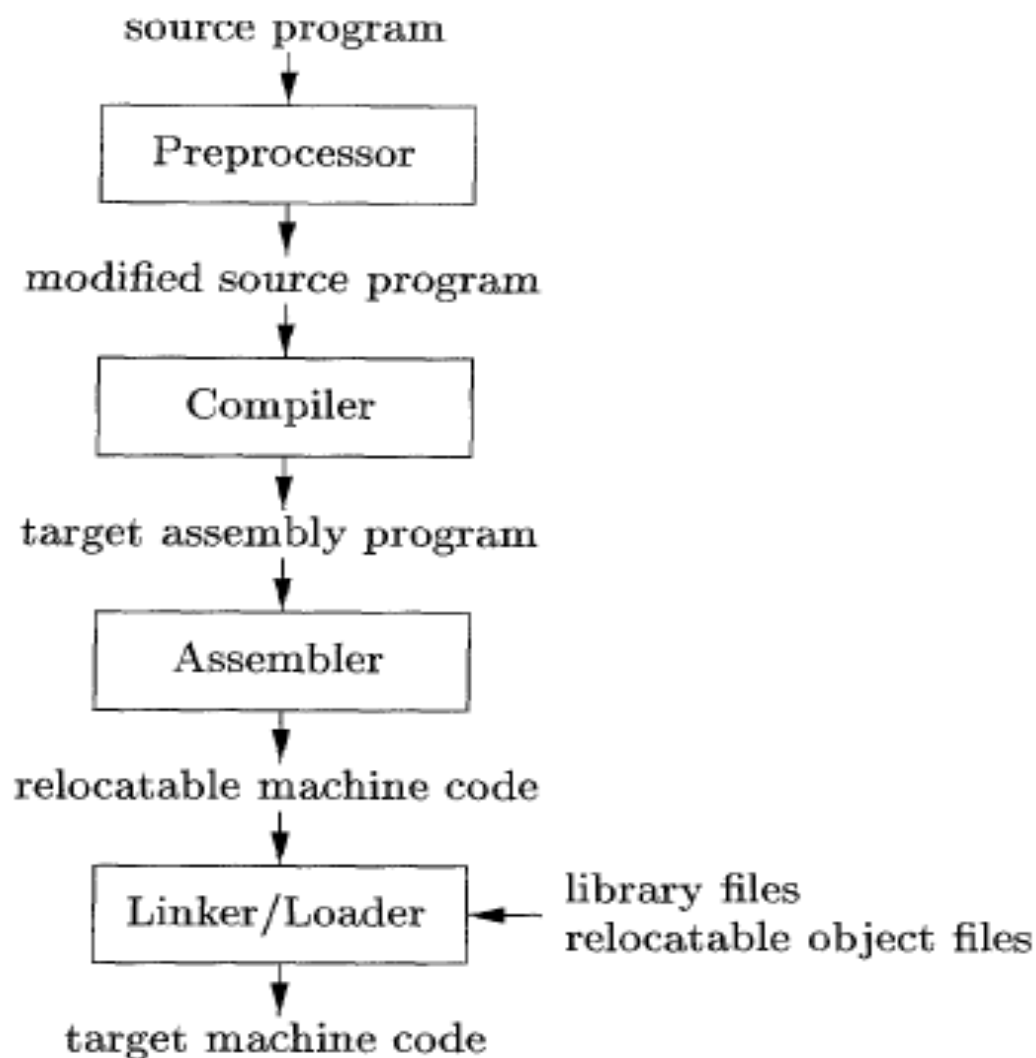
- Компилаторите генерират програми, които се изпълняват много по-бързо в сравнение с програмите, изпълнение от интерпретатори.
- Интерпретаторите предоставят по-добра диагностика на грешки в сравнение с компилаторите, защото изпълняват програмата оператор по оператор.

Хибриден компилатор

Хибридните компилатори комбинират компилиране и интерпретиране. Пример за хибриден компилатор е езиковият процесор на Java. Входната програма първо се компилира до междинен код, нар. *байт код (bytecode)*. Байт кодът в последствие се интерпретира от виртуална машина. Основната полза е, че веднъж компилиран на една машина, байт кодът може да бъде изпълняван на всяка друга машина, разполагаща с виртуалната машина на Java.



Обща схема на езиков процесор



За създаване на изпълнима изходна програма са нужни следните системни средства (програмни инструменти):

- Препроцесор (Preprocessor) - Препроцесорът обединява програмен код от различни модули, съхранявани като отделни файлове. Може да преобразува кратки програмни фрагменти, нар. макроси, в оператори на входния език. Препроцесорът модифицира входната програма.
- Компиляторът – Компиляторът генерира програмен код на асемблер, защото асемблерният код е по-лесен за генериране и за трасиране (debug).
- Асемблер (Assembler) – Генерира машинен код, който е преместваем в адресното пространство (относителни адреси).
- Свързваща програма (Linker/Loader) - По-големите програми обикновено се компилират на отделни части. В резултат се налага преместваемият машинен код да бъде свързан с други преместваеми обектни и библиотечни файлове за получаване на код за изпълнение от дадена машина. Свързващата програма (linker) изпълнява т.нар. резолиране на външни адреси, т.е. насочва адресите от даден модул към външни модули. Зареждащата програма (loader) позиционира всички изпълними обектни файлове в паметта за изпълнение.

Структура на компилатор

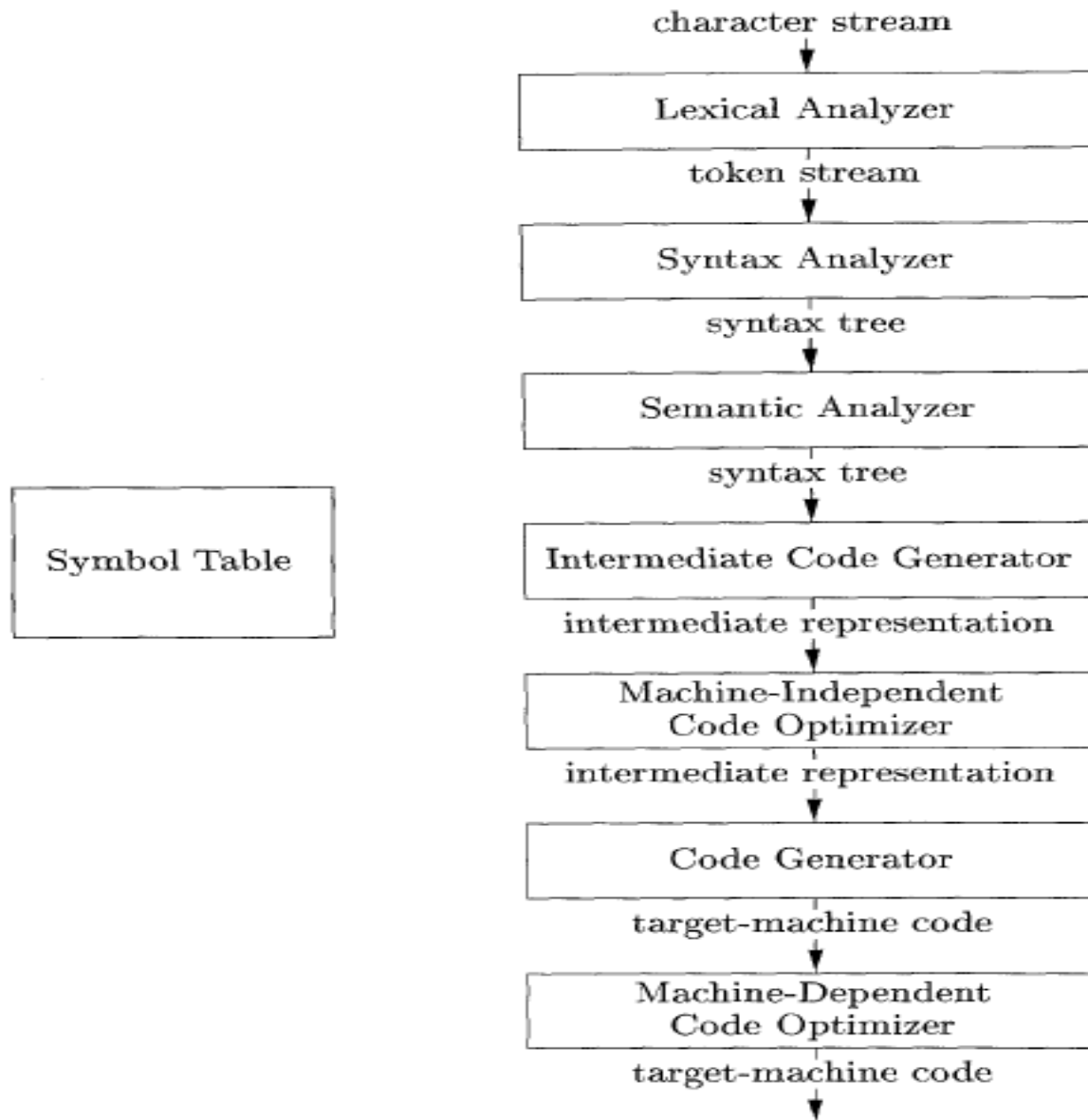
Процесът на компилиране условно преминава през два етапа : *анализ* и *синтез*.

- **анализ** - Проверява входната програма за съответствие с граматическата структура на езика и създава междинно представяне (междинна форма) на програмата. По време на анализа също се събира информация за входната програма, която се съхранява в т.нар. **символна таблица**. Тази таблица заедно с междинната форма се изпраща за синтез.
- **синтез** - Синтезът построява изходна програма от получените междинна форма и символна таблица

Често анализът се нарича **front end** на компилатора, а синтезът **back end**.

- Разгледан в детайли, процесът на компилиране се изпълнява като последователност от етапи, във всеки от които едно представяне (една форма) на входната програма се трансформира (преобразува) в друго представяне (друга форма).
- На практика няколко етапа мога да бъдат групирани (обединени).
- Символната таблица, която съхранява информация за цялата входна програма, се използва във всички етапи на компилатора.

Етапи на компилирането



Лексически анализ

Лексическият анализатор чете поток от символи, съставлящи входната програма и обединява символите в поток от значещи последователности, нар. лексеми или думи (tokens). За всяка лексема лексическият анализатор генерира двойка стойности:

<клас на лексемата(token-name), стойност(attribute-value)>

Клас на лексемата се използва при синтактичния анализ

Стойност сочи към елемент от символната таблица. Информацията от символната таблица се използва при семантичния анализ и генериране на код.

Потокът от лексеми се изпраща към следващия етап – синтактичен анализ.

Лексемите се наричат също *символи на речника на езика*.

Примери за лексеми, разпознавани на етапа на лексическия анализ:

- Идентификатори – последователности от букви и цифри
- Числа – последователности от цифри
- Оператори/разделители – последователности от специални символи

Синтактичен анализ

- Потокът от лексеми се представя във форма, която отразява синтактичната структура на входната програма и позволява лесно разпознаване на тази структура. Този етап се нарича *синтактичен анализ* (или *парсиране - parsing*) .
- Синтактичният анализатор (parser) използва класовете лексеми, получени от лексическия анализатор, за построяване на дървовидно представяне на входната програма, описващо нейната граматическа структура.
- Типичното представяне е във вид на синтактично дърво, в което даден представя операция, а неговите наследници листа представляват аргументите (операндите) на операцията.
- Полученото синтактично дърво, отразяващо граматическата структура, се използва от следващите етапи на компилатора за семантичен анализ и генериране на изходна програма.

Семантичен анализ

- Семантичният анализатор използва синтактичното дърво и информацията от символната таблица, за да провери дали семантиката (смисълът) на входната програма съответства на дефинициите на входния език. Семантичният анализатор също събира информация за използваните типове данни и я съхранява в синтактичното дърво или в символната таблица за последващо използване при генерирането на код.
- Важна част от семантичния анализ е проверката за съвместимост на типове, при която компилаторът проверява дали всеки оператор има операнди от правилен тип. Пример: В повечето програмни езици индексите на масив трябва да са целочислени стойности. Следователно компилаторът трябва да регистрира грешка, ако за индекс се използва нецяло число.

Генериране на междинен код

След синтактичен и семантичен анализ на входната програма много компилатори генерират междинно представяне на програмата, което се нарича **код за абстрактна (виртуална) машина**. Този код трябва да изпълнява две важни изисквания:

- Трябва да бъде лесен за генериране;
- Трябва да бъде лесен за транслиране в код за дадена реална машина.

Едно често използвано междинно представяне е три адресен код. Той представлява последователност от асемблерни инструкции с три операнда за всяка инструкция.

Оптимизация на кода

- Оптимизацията на получения машинно независим код се опитва да подобри междинния код по отношение на различни показатели – най-често бързодействие или краткост.

Генериране на код

- Генераторът на преобразува междинното представяне на входната програма в програма на **изходен език**. Ако изходният език е машинен, за всяка променлива от входната програма се избират регистри или адреси от паметта. След това инструкциите от междинния код се транслират в последователности от машинни инструкции със същото действия. Критичен момент при генерирането на код е правилно разполагане на стойностите на променливите в съответни регистри.

Управление на символните таблици

- Основна функция на компилатора е да съхранява имената на променливите, използвани във входната програма и да събира информация за атрибутите на всяка променлива – тип, област на видимост (scope), адресно пространство и др.
- Символната таблица е структура, която съдържа запис за всяко име на променлива с полета за съхраняване на атрибутите на променливата. Тази структура трябва да бъде създадена така, че компилаторът да намира бързо всеки запис от нея, както и бързо да записва и прочита данни.

Обединяване на етапи в компилатора

- Описаните етапи засягат логическата организация на един компилатор.
- При реализацията операциите от няколко етапа могат да бъдат обединени в един „пас“, при който се прочита входна програма и се генериране изходна програма.
- Например front-end етапите на лексически, синтактичен и семантичен анализ и генериране на междинен код могат да бъдат обединени в един пас. Оптимизацията на кода може да бъде незадължителен пас. След това може да има отделен Back-end пас за генериране на код за дадена машина.

Формално определение на
езиците за програмиране.
Граматики.Класификация.
Бакус-Наур форма

Азбука

- *Азбука* се нарича крайно множество от символи (букви). Примери за символи от азбука са букви, цифри, пунктуационни знаци.

Примери:

- Множеството $\{0,1\}$ представлява *двоична азбука*.
- *ASCII* е пример за азбука, използвана в множество приложения и системи.
- *Unicode* включва около 100,000 символи от различни езици.
- *Низ* или *дума* (*string*) над дадена азбука се нар. крайна последователност от символи от азбуката. Дължина на низ s , отбелязвана с $[s]$, се нар. броят на символите в s .

Примери:

- **banana** е низ с дължина 6.
- *Празен низ* (*empty string*) се отбелязва с ϵ е низ с дължина 0.

Формален език

- Ако x и y са низове, конкатенацията на x и y , означена като xy , е низ, получен чрез съединяване на x и y . Например, ако $x = \text{dog}$ и $y = \text{house}$, тогава $xy = \text{doghouse}$. За празния стринг $\varepsilon s = s\varepsilon = s$.
- **Език** се нар. крайно множество от низове (думи) над дадена азбука. Това определение е много широко.

Формална граматика

- **Граматика** е средство за описание и анализ на формални езици. Състои се от **правила**, които описват как се получават валидни изречения на езика.
- Пример от граматиката в английския език:
изречение → <подлог><глаголна_фраза><допълнение>
подлог → This | Computers | I
глаголна_фраза → <наречие> <сказуемо> | <сказуемо>
наречие → never
сказуемо → is | run | am | tell
допълнение → the <същ._име> | a <същ._име> | <същ._име>
същ._име → university | world | cheese | lies

Правилата се нар. още *продукции*. Стрелката в правило се чете като „поражда“.

Чрез прилагане на правилата се получават изречения:

This is a university.

Computers run the world.

I am the cheese.

I never tell lies.

Пример за прилагане на правилата за получаване на горното изречение.

sentence → <subject> <verb-phrase> <object>

→ This <verb-phrase> <object>

→ This <verb> <object>

→ This is <object>

→ This is a <noun>

→ This is a university

Пример 2: Операторът **if-else** в Java има следната форма

if (израз) оператор **else** оператор

Тази форма означава, че операторът **if-else** се получава чрез конкатенация (съединяване) на ключовата дума **if** с отваряща скоба, израз, затваряща скоба, оператор, ключовата дума **else** и друг оператор. С използване на променливата *expr*, означаваща израз и променливата *stmt*, означаваща оператор, горното правило може да се запише по следния начин:

$stmt \rightarrow \mathbf{if} \ (\ expr \) \ stmt \ \mathbf{else} \ stmt$

В правилата елементи като ключовата дума **if** и скобите се нар. **терминални символи (терминали)**. Променливи като *expr* и *stmt* представляват последователности от терминални символи и се нар. **нетерминални символи (nonterminals)**.

Дефиниции

- *граматика* - множество от правила за получаване на валидни изречения от езика.
- *нетерминал* – символи от граматиката, които могат да бъдат заместени (разширени) с други символи.
- *терминал* - елемент от езика; **не мога да бъдат заместени с никакви други символи.** След получаване на терминален символ не може да бъде приложено следващо правило.
- *Правило (продукция)* – правило от граматиката за заместване на символи. Общата форма за прилагане на правило с нетерминали е:

$$X \rightarrow Y_1 Y_2 Y_3 \dots Y_n$$

Нетерминалът X е еквивалентен на конкатенацията от символи $Y_1Y_2Y_3\ldots Y_n$. Правилото означава, че при всяко срещане на X , то може да се замени с последователност $Y_1Y_2Y_3\ldots Y_n$. Когато се получи последователност от символи, в която никой от символите не може да се замени с друг, това означава, че тези символи са термини. Такъв низ се нар. *изречение*. **В термините на програмните езици изречението представлява синтактично коректна, завършена програма.**

- *извеждане (derivation)* – такава последователност от прилагане на правилата, при която се получава крайна последователност от термини. *Ляво извеждане (left-most derivation)* имаме, когато винаги замествахме най-левия нетерминал при прилагане на правилата. Възможно е и *дясно извеждане (right-most derivation)*.

- *стартов символ* - нетерминален символ, от който започва прилагането на правилата (извеждането):

$$S \rightarrow X_1X_2X_3\dots X_n$$

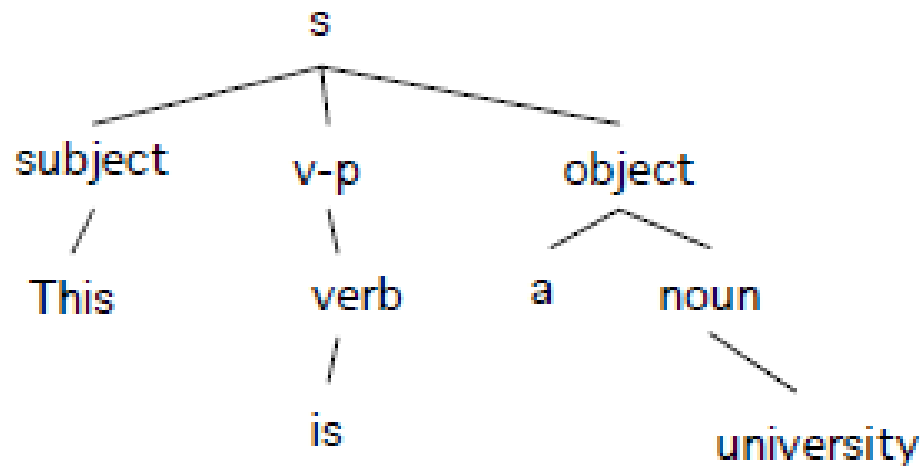
Всички изречения на езика се извеждат от S чрез последователно заместване на символи в съответствие с правилата.

- *празен символ ε* - понякога се налага даден символ да бъде заменен с празния символ. Например: $A \rightarrow B \mid \varepsilon$.
- *БНФ (Бакус-Наур форма)* - начин за описание на програмни езици чрез формални граматики и правила.

Представяне на извеждането

Прилагането на правилата за извеждане на изречение може да се представи по два начина:

- Първият начин е извеждане, при което правилата се прилагат стъпка по стъпка и на всяка стъпка се замества нетерминал.
- Вторият начин е чрез дърво. То показва как се заместват символи в йерархичен вид. Пример за извеждане на изречението "This is a university":



Първият начин показва реда, в който са приложени правилата за извеждане на изречение. Дадено изречение може да бъде изведено с различни последователности от прилагане на правилата (напр. ляво или дясно извеждане).

Дървовидното представяне на показва реда на прилагане на правилата за получаване на изречение. На дадено изречение съответства единствено дърво.

Формално определение за граматика

Грамматика $G = (V_T, V_N, P, S)$ се състои от четири компонента:

1. Множество от терминални символи (V_T)
2. Множество от нетерминални символи (V_N)
3. Множество от правила (P)
4. Стартов символ (S).

Зависимости

$$V_T \cap V_N = \emptyset$$

$$V = V_T \cup V_N$$

Ако имаме двойка символи (α, β) , където

$\alpha \in V^+$, $\beta \in V^*$, тогава

$\alpha \rightarrow \beta$ е правило от P

Стартов символ $S \in V_N$

където

V^* - множество от всички низове **плюс** празния низ ϵ .

V^+ - множество от всички низове **без** ϵ .

Обобщения

Граматиките се задават чрез *правила (продукции)*, като първото правило е за стартовия символ.

Цифрите, символи като $<$, $\leq$, while, if и др. са терминали. Обикновено с главна буква се означава нетерминал, а с малки букви се означават терминали.

Правилата могат да се обединяват чрез специалния символ “|”, който означава „ИЛИ,,.

Пример:

Граматика, която поражда низове от език $\{a^m b^n \mid m, n \geq 0\}$ се дефинира по следния начин:

$$G_1 = (\{a, b\}, \{S, A, B\}, P, S)$$

$$S \rightarrow AB$$

$$A \rightarrow aA$$

$$A \rightarrow \varepsilon$$

$$B \rightarrow bB$$

$$B \rightarrow \varepsilon$$

Низ **aaabb** се поражда от тази граматика чрез прилагане на следните правила:

$$S \rightarrow AB \rightarrow aAB \rightarrow aaAB \rightarrow aaaB \rightarrow aaabB \rightarrow aaabb$$

Еквивалентност

Език $L(G)$, описан чрез граматика G , е множество от изречения, породени от граматиката G .

Изреченията на даден език могат да бъдат породени от повече от една граматика.

Две граматики G и G' са *еквивалентни*, ако езиците, $L(G)$ и $L(G')$, които пораждат, са еднакви.

Пример: Граматика, която е еквивалентна на граматиката G_1 :

Еквивалентност

$$G_2 = (\{a, b\}, \{S, Y\}, P, S)$$

$$S \rightarrow aS \qquad Y \rightarrow b$$

$$S \rightarrow a \qquad Y \rightarrow bY$$

$$S \rightarrow b \qquad Y \rightarrow \varepsilon$$

$$S \rightarrow bY$$

$$S \rightarrow Y$$

Извеждане на низ aaabb:

$$S \rightarrow aS \rightarrow aaS \rightarrow aaaS \rightarrow aaabY \rightarrow aaabb$$

$$G_2 \equiv G_1$$

Горните правила могат да бъдат обединени с използване на символ ИЛИ - “|” :

$$S \rightarrow aS \mid a \mid b \mid bY$$

$$Y \rightarrow b \mid bY \mid \varepsilon$$

Йерархия на граматики

- Американският лингвист Ноам Хомски е извършил солидни изследвания върху формални граматики.
- В йерархията (класификацията) на Хомски съществуват четири типа формални граматики, започващи от тип 0 (най-обща) до тип 3 (най-рестриктивна).
- Ако граматиката има повече ограничения към правилата, тя е по-лесна за описание и реализация, но е с по-малка мощност (т.е. поражда по-малко низове в езика).

Йерархия на Хомски

- Тип 0: **без ограничения**. Този тип граматики са най-общи. Правилата са във вида $u \rightarrow v$, където u и v са произволни низове от V . Няма ограничения към правилата, освен че в лявата страна на правило не може да има празен низ.
- Тип 1: **контекстно-зависими граматики (context-sensitive)**. Правилата са във вида $uXw \rightarrow uvw$, където u , v и w са произволни низове от символи от V , където v не е празен низ, а X е единствен нетерминал. Това означава, че X може да бъде заместен от v , само ако е обграден от символи u и w . (т.е. в конкретен контекст).

Йерархия на Хомски

- Тип 2: **контекстно-свободни граматики (context-free)**. Правилата са във вида $X \rightarrow v$, където v е произволен низ от символи от V , а X е единствен нетерминал. Навсякъде, където се среща X , той може да бъде заместен с v (независимо от контекста).
- Тип 3: **регулярни граматики**. Правилата са във вида $X \rightarrow a$, $X \rightarrow aY$, or $X \rightarrow \epsilon$, където X и Y са нетерминали, a е терминал. Т.е., лявата страна трябва да е единствен нетерминал, а дясната страна трябва да е празен низ, единствен терминал или единствен нетерминал. Тези граматики са най-ограничени по отношение на брой породени символни низове.

Йерархия на Хомски

Граматика от тип 3 (G_3) са най-лесни за обработване поради липса на рекурсивни правила.

За някои класове граматика от тип 2 (G_2) съществуват ефективни анализатори.

Въпреки че граматика от тип 1 (G_1) и тип 0 (G_0) са по-мощни от G_2 и G_3 , те имат по-малко практическо приложение поради невъзможността на създаване на ефективни анализатори за тях.

При създаване на транслатори на програмни езици най-често се използват контекстно-свободни граматика (G_2), които обикновено се означават като CFG (Context-Free Grammars).

Регулярни изрази

Регулярни изрази – правила за точно дефиниране на множества от низове, които са валидни в даден формален език. Правилата се задават със следните три оператора:

- Съединяване (конкатенация) xu
- Алтернатива $x|y$ x или y
- Повторение x^* x се повтаря 0 или повече пъти

Формално регулярните изрази се дефинират чрез следните рекурсивни правила:

- 1) Всеки символ от азбуката на формален език е регулярен израз
- 2) ϵ е регулярен израз
- 3) Ако r_1 и r_2 са регулярни изрази, то (r_1) , r_1r_2 , $r_1 | r_2$, r_1^* са също регулярни изрази
- 4) Никакъв друг израз не е регулярен

Регулярни изрази

Чрез регулярни изрази мога да бъдат дефинирани лексеми (думи) в програмните езици.

Пример за регулярен израз за дефиниране на цяло число, съставено от една или повече цифри:

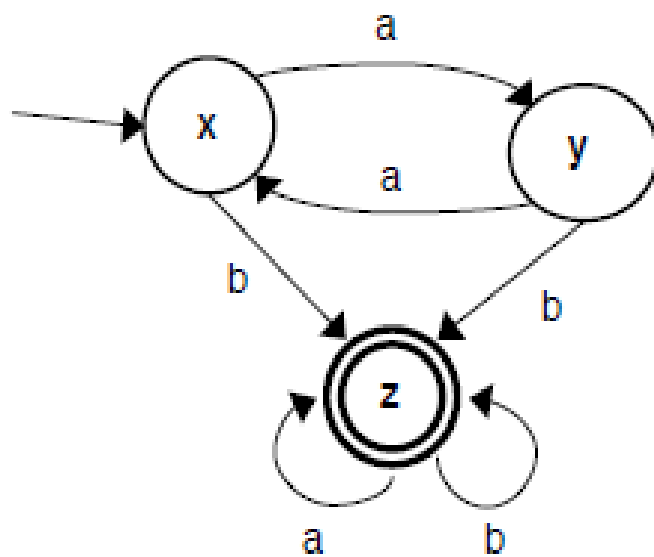
(+ е специален символ в регулярния израз, който означава 1 или повече повторения)

$(0|1|2|3|4|5|6|7|8|9)^+$

Крайни автомати

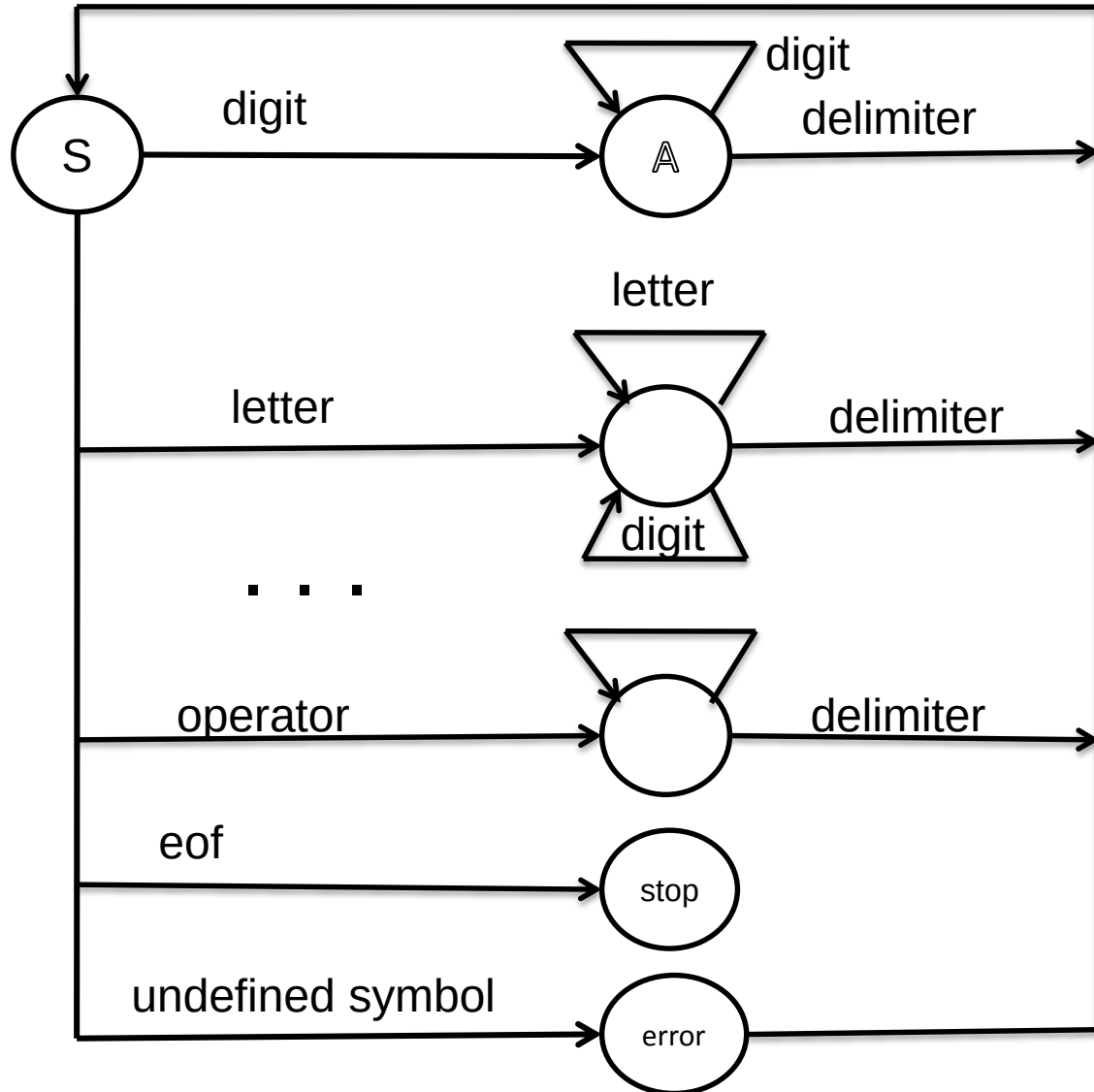
След като лексемите са дефинирани чрез регулярни изрази, можем да създадем краен автомат за тяхното разпознаване. *Крайният автомат (finite automata)* се състои от:

- 1) Краен брой състояния, едно от които е начално, а някои състояния (или нито едно) са крайни.
- 2) Азбука Σ от възможни входни символи.
- 3) Краен брой преходи, които дефинират за всяко състояние при даден символ от азбуката кое е следващото състояние.

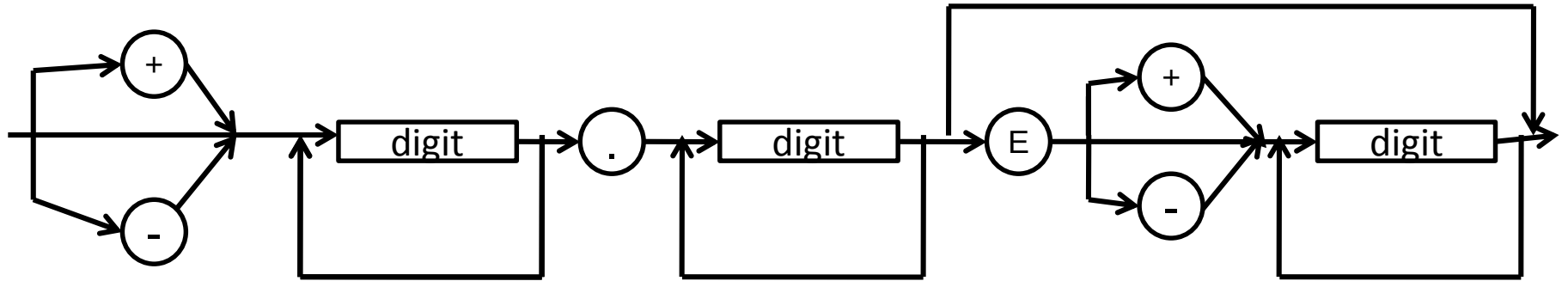


		a	b
(start)	x:	y	z
	y:	x	z
(final)	z:	z	z

Краен автомат за разпознаване на лексеми в Pascal

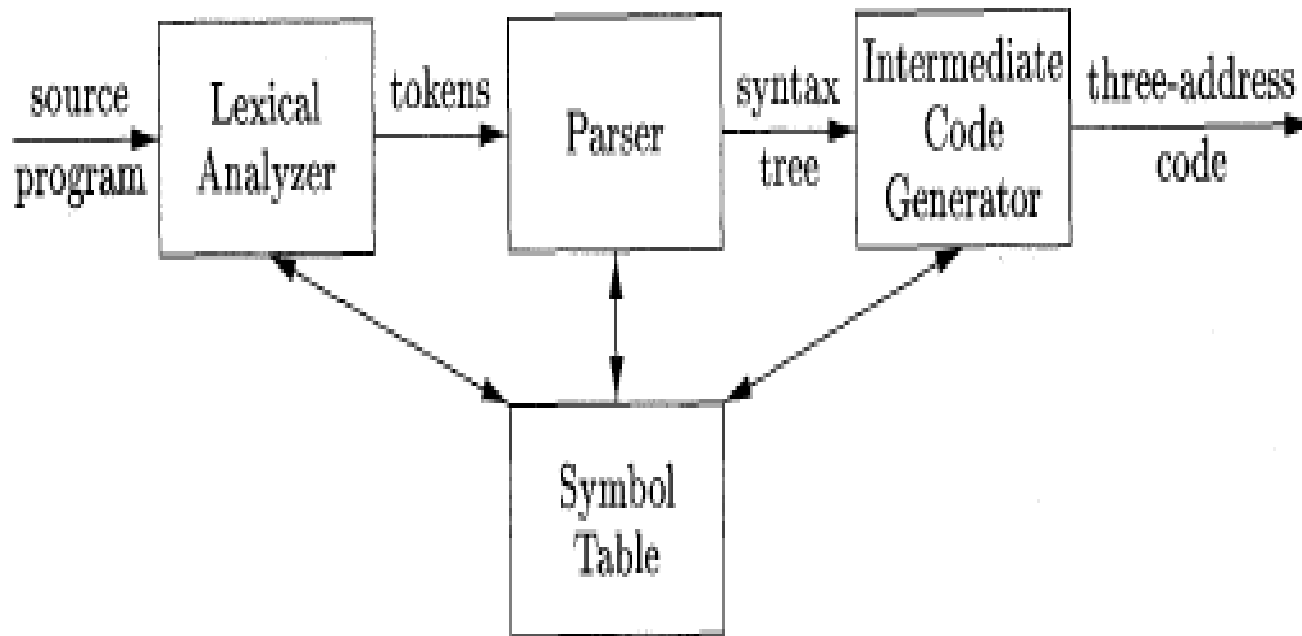


A in more details



Лексически анализ

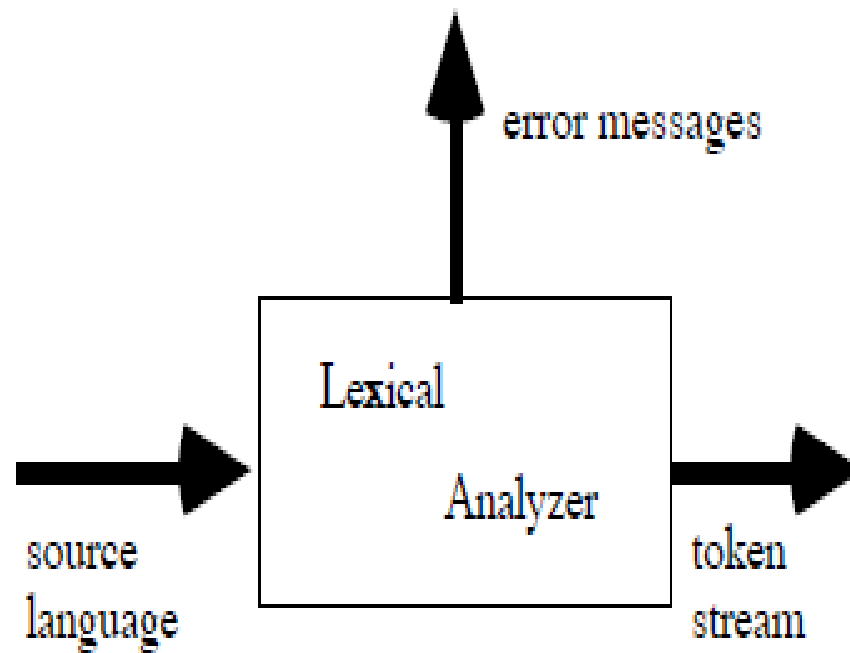
Обща организация на компилатор



ОСНОВНИ ПОНЯТИЯ

- *Лексически анализ* или *сканиране* е процес, при който входната програма се прочита отляво надясно и се формират символни последователности (нар. *tokens*).
- *Tokens* са последователности от символи, които са значещи в програмния език. Даден програмен език типично съдържа няколко групи такива последователности: константи (цели числа, реални числа, символи, символни низове и др.), оператори (аритметични, логически, за сравнение и др.), разделители, ключови (резервирани) думи.

```
while (i > 0)
  i = i - 2;
```



```
while
(
  i
>
  0
)
i
=
...
```

Лексическият анализатор чете входна програма, а на изхода се получава поток от символни последователности. Примери за символни последователности, които се разпознават на етапа на лексическия анализ:

3 или **255** е целочислена константа

"Fred" или **"Wilma"** е символен низ

numTickets или **queue** е име на променлива

Стойностите на горните последователности се нар. *лексему*.

За някои типове лексеми има само по една съответстваща лексема (напр. >), а други имат множество лексеми (напр. целочислени константи).

Целта на лексическия анализатор е да раздели входната програма на валидни последователности на входния език, но не може да определи дали всяка лексема е на правилното място.

- По време на лексическия анализ могат да бъдат открити малко на брой типове грешки:
 - невалидни (или неразпознати) лексеми
 - неправилни лексеми (напр. грешен брой апострофи в символна константа, недовършени коментари и др.)

По време на лексическия анализ не могат да бъдат открити лексеми, които не са на правилно място, нито недекларирани идентификатори, сгрешени ключови думи, несъвместими типове и др.

Например лексическият анализатор няма да открие грешки в следната входна последователност, защото той все още „не знае“ правилата за подредба на лексемите. Тези грешки се откриват на по-късния етап на синтактичен анализ.

```
int a double } switch b[2] =;
```

Освен това лексическият анализатор „не знае“ как трябва да бъдат групирани разпознатите лексеми. В горната последователност ще бъдат открити лексеми **b**, **[**, **2** и **]**, т.е. четири отделни лексеми, които всъщност заедно формират достъп до елемент на масив.

Лексическият анализатор е подходящ за изпълнение и на някои други задачи като например:

- Премахване на коментари, интервали, табулации;
- Макроси, условно компилиране.

Лексически анализ

```
void main () {  
    bool NotEndOfSource = true;  
    do {  
        // read next input character  
        // classify character by symbol group  
        // build symbol  
        // write intermediate code  
    } while (NotEndOfSource);  
}
```

Лексически анализ

```
#include <stdio.h>

void main () {
    bool NotDone = true
    char ch;
    do {
        ch = getchar ();
        switch (ch) {
            case 'a': case 'b': /* . . . */ case 'z' :
                /* scan identifier */
                break;
            case '0': case '1' : /* . . . */ case '9':
                /* scan number */
                break;
```

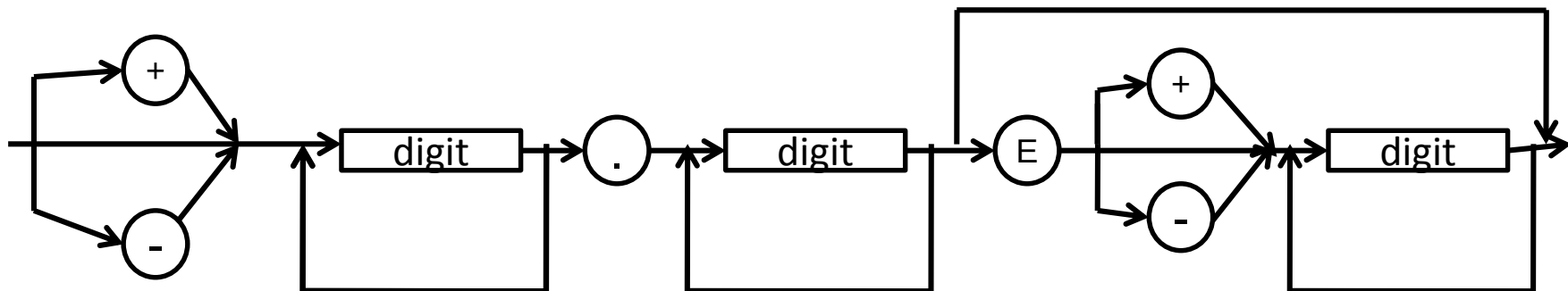
Лексически анализ

```
case '<':
    /* scan operator < */
    break;
    /* . . . */
case ' ':
    /* scan white spaces */
    break;
case '.':
    NotDone = false;
    break;
};
} while (NotDone);
}
```

Премахване на интервали

```
#include <stdio.h>

void main() {
    char  ch;
    /* skip white spaces */
    while ( (ch = getchar() ) == ' ');
    /* . . . */
}
```



Разпознаване на дробна част в реални числа

```
#include <stdio.h>
void main() {
    char  ch;
    ch = getchar();
    while ( (ch>='0') || (ch <= 9) ) {
        // collect the integer part
    };
    if ( ch == '.' ) {
        // collect the fractional part
    };

    if ( ch == 'e' ) {
        // collect the exponent part
    };
    // build the number
    // write intermediate code
}
```

Разпознаване на цяла част в реални числа

```
#include <stdio.h>
extern "C" void error (char *);
void main() {
    float number = 0.0;
    char ch
    const float float_limit = float (0xFFFFFFFFF);
    ch = getchar ();
    while ( ( ch = '0' ) || ( ch <= '9' ) ) {
        if ( number < float_limit ) {
            number = number * 10 + ( ch - '0' );
            ch = getchar ();
        } else {
            error ("float exceeded \n");
        }
    }
}
```

Символни таблици

Символните таблици са структури от данни, които се използват от компилаторите за съхраняване на информация за използваните конструкции във входната програма. Тази информация се формира постъпателно по време на анализа и се използва по време на синтеза за генериране на изходен код.

Елементите в символната таблица съдържат например следната информация за идентификатор: име (т.е. лексемата), тип на данните, позиция в паметта и др.

Символните таблици типично поддържат множество декларации на един идентификатор в програмата.

Елементи в символна таблица:

Различни типове на имената

- Променливи (идентификатори)
 - име на променливата (лексема), може да има ограничение в броя на символите
 - тип на данните,
 - име и ниво на блок, в който е декларирана променливата
 - друга информация за права на достъп
 - базов адрес и отстъп в паметта

Записи в символната таблица:

Различни типове данни

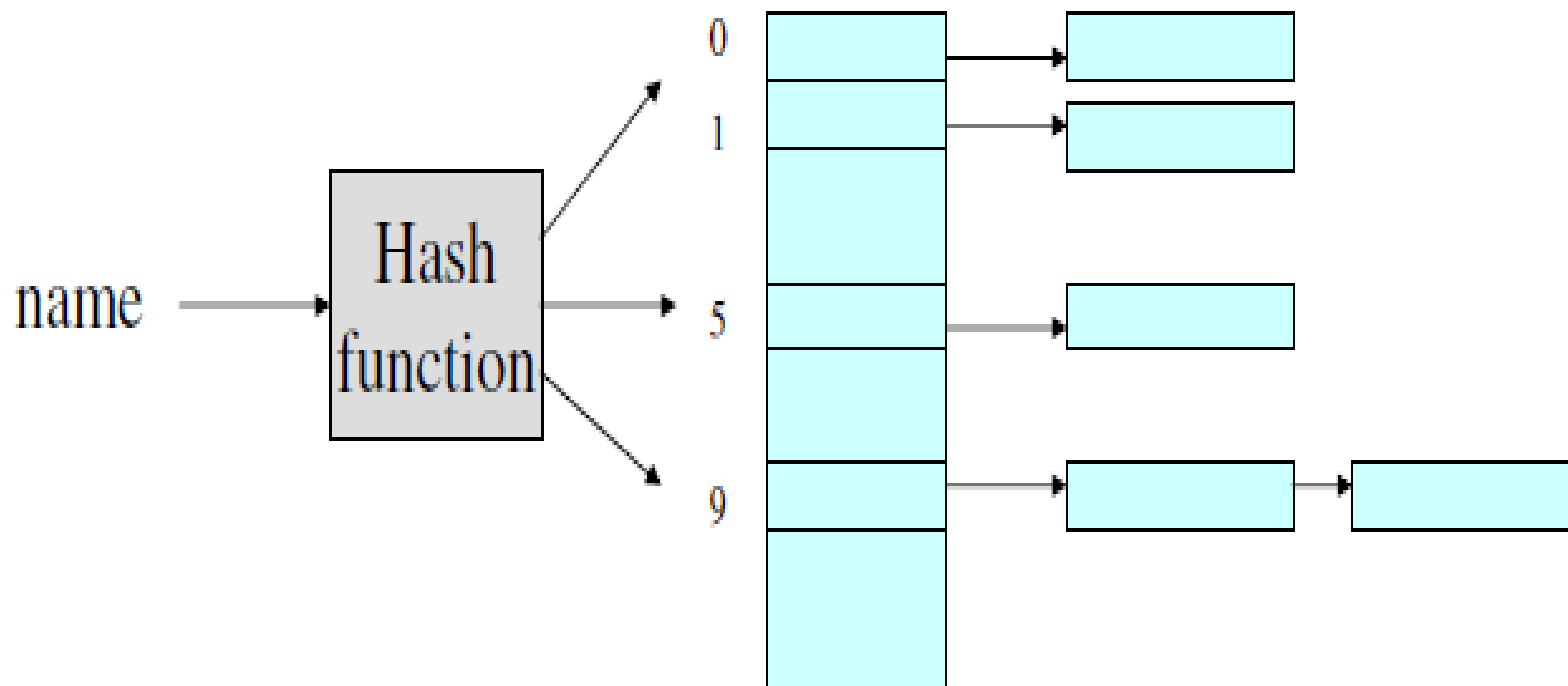
- Масиви
 - Размерности
 - Горна и долна граница на всяка дименсия
- Записи и структури
 - Списък от полета
 - Информация за всяко поле
- Функции и процедури
 - Брой и тип на параметрите
 - Тип на върнатата стойност

Операции със символната таблица

Двете основни операции със символната таблица са:

- Въвеждане на нов ред – т.е. запис на нов идентификатор (име на променлива, тип и др.)
 - Търсене на запис – т.е. търсене на информация по зададено име
- Търсенето се извършва много по-често от въвеждането
 - За представяне на символна таблица най-често се използват хеш таблици поради възможностите за ефективно търсене

Представяне на символна таблица



Последователно търсене

```
void main () {  
    const N = 769;  
    char * a[N];  
    char * X = “”;  
    int i;  
    // build the identifier table a[]  
    // put current identifier into “X”  
    i = -1;  
    do {  
        i += 1;  
    } while ( ( a[i] != X ) && ( i != N-1 ) );  
    If ( a[i] != X ) { /* element not found */  
}
```

ДВОИЧНО ТЪРСЕНЕ

```
#include <string.h>
```

```
void main () {
```

```
    const N = 769;
```

```
    char * a[N];
```

```
    char * X = "";
```

```
    int i, j, k;
```

```
    // build and sort identifier table a[]
```

```
    // put current identifier into "X"
```

```
    i = 0;
```

```
    j = n - 1;
```

Двоично търсене

```
do {  
    k = (i+j)/2;  
    If ( strcmp (X, a[k])) < 0  
        i = k + 1;  
    else  
        if ( strcmp (X, a[k])) > 0  
            j = k -1;  
} while ( (a[k] != X) && (i <= j) );  
If ( a[k] != X ) { /* element not found */ };  
}
```

Генериране на хеш ключ

```
#include <string.h>
const int N = 769;
int Hash (char * id) {
    unsigned int key, i;
    char c;
    key = 1;
    i = -1;
    do {
        i += 1;
        c = id[i];
        if ( c != '\0' )
            key = (key + c) % N;
    } while ( ( c != '\0' ) && ( i != strlen(id) ) )
    return (key);
}
```


Търсене с хеш ключ

```
void main () {  
    int index;  
    char * identifier = "asdasd";  
  
    /* . . . */  
  
    index = Hash (identifier);  
  
    /* . . . */  
}
```

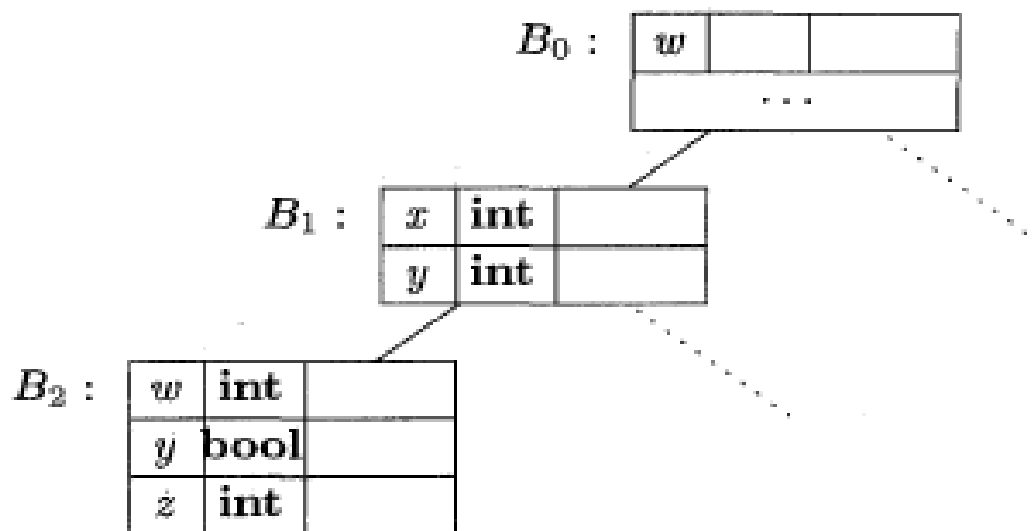
Всяка променлива се декларира с област на действие даден фрагмент от програма (т.нар. scope). Често за всеки такъв фрагмент се реализира отделна символна таблица. Това означава, че даден фрагмент от програмата има собствена символна таблица, в която има ред за всяка декларация.

Блокове

- Областта на действие на име (идентификатор) е свързана с **блоковата структура** на повечето програмни езици:
 - Стандартни блокове (съставен оператор, оператор if)
 - Процедури и функции
 - Програма (глобален блок)
- Имената трябва да бъдат уникални в рамките на блока, в който са декларирани, т.е. не може да има две еднакви имена в един блок.
- Резолиране на имена: имена, декларирани във външен блок могат да бъдат използвани във вътрешни блокове.
- Същото се отнася и за други конструкции; например клас в обектно-ориентирана програма има собствена таблица с отделен ред за всяко поле и за всеки метод.

Свързани символни таблици

Вложени блокове се реализират чрез свързани символни таблици, т.е. таблицата на даден блок сочи към таблицата на обхващайкия блок.



Синтактичен анализ

На етапа на синтактичен анализ компилаторът проверява дали последователността от лексеми (tokens), получени при лексическия анализ, представляват валидно изречение в граматиката на програмния език.

Съществуват два основни метода за синтактичен анализ:

- *Отгоре-надолу (top-down parsing)* - започваме от стартовия символ и прилагаме правилата до достигане на търсения символен низ.
- *Отдолу-нагоре (bottom-up parsing)* - започваме от символния низ и го редуцираме чрез прилагане на правилата до достигане на стартовия символ.

Примери

Дадена е граматика G , която разпознава низове, съдържащи произволен брой a , следвани от поне едно (или повече) b :

$$S \rightarrow AB$$

$$A \rightarrow aA \mid \varepsilon$$

$$B \rightarrow b \mid bB$$

Пример: отгоре-надолу (*top-down*)

Анализ отгоре-надолу за пораждање на низ **aaab**:

Започваме със стартовия символ и на всяка стъпка замествахме един от нетерминалните символи в низа с лявата страна на някое от правилата. Повтаряме това до получаване на низ, съставен само от терминални символи. При анализа отгоре-надолу низът (изречението) се получава отляво надясно.

S

AB $S \rightarrow AB$

aAB $A \rightarrow aA$

aaAB $A \rightarrow aA$

aaaAB $A \rightarrow aA$

aaaεB $A \rightarrow \epsilon$

aaab $B \rightarrow b$

Пример: отдолу-нагоре (*bottom-up*)

Анализът отдолу-нагоре работи в обратен ред. Започваме с изречение от терминални символи и на всяка стъпка прилагаме правило на обратно (т.е. отдясно-наляво), замествайки част от низа с нетерминален символ в лявата част на правилото. Продължаваме до получаване на стартовия символ в граматиката. При този анализ изречението се получава отдясно наляво.

aaab

aaaεb (insert ε)

aaaAb $A \rightarrow \epsilon$

aaAb $A \rightarrow aA$

aAb $A \rightarrow aA$

Ab $A \rightarrow aA$

AB $B \rightarrow b$

S $S \rightarrow AB$

При създаване на синтактичен анализатор в компилатора обикновено са наложени ограничение за обработване на входа. В разгледаните примери е лесно да решаваме кое правило да приложим на всяка стъпка, защото виждаме целия низ $aaab$.

На практика, обаче, входният низ се получава като поток от символи, т.е. символ по символ. **Обикновено сме ограничени да виждаме само един символ напред.** Виждаме само следващия символ във входния поток (*lookahead*). Това ограничение създава проблем при реализацията на синтактичен анализатор.

Например в разгледаната граматика, ако анализаторът „вижда“ само пореден прочетен символ b , той не може да „избере“ точно кое правило да приложи.

$B \rightarrow b$ or $B \rightarrow bB$.

Връщане назад (backtracking)

Едно от възможните решения за реализация на синтактичен анализ е връщане назад (*backtracking*). При него анализаторът избира да приложи едно правило. Ако след този избор се окаже, че заместването на символите не може да продължи (т.е. няма подходящо правило), анализаторът се връща по входния низ и започва отново, избирайки различно правило.

Пример:

Дадена е следната проста граматика:

$$S \rightarrow bab \mid bA$$
$$A \rightarrow d \mid cA$$

Разглеждаме синтактичен анализ на изречение `bcd`.

Лявата колона показва полученият низ на всяка стъпка, в средната колона е оставащият входен поток, а в дясната колона в правилото, което се прилага на съответната стъпка.

S	bcd	Опитваме $S \rightarrow bab$
bab	bcd	съвпада b
ab	cd	блокировка (dead-end), връщане назад
S	bcd	опитваме $S \rightarrow bA$
bA	bcd	съвпада b
A	cd	опитваме $A \rightarrow d$
d	cd	dead-end, връщане назад
A	cd	опитваме $A \rightarrow cA$
cA	cd	съвпада c
A	d	опитваме $A \rightarrow d$
d	d	съвпада d

Успех!

При всяко срещане на dead-end се връщаме до последното приложено правило, където прилагаме различно правило. Ако всички алтернативни правила са били проверени и водят до dead-end, се връщаме до предишното правило и т.н. Това продължава до получаване на входния низ или до изчерпване на всички комбинации от правила без успех.

Анализ отгоре-надолу с предсказване

Фокусираме се върху анализ отгоре-надолу. Ще разгледаме начин за реализация на синтактичен анализатор без връщане назад, който се нарича *предсказващ*.

Предсказващият анализатор избира кое правило да приложи само на базата на следващия входен символ и на текущо обработвания нетерминал. За да бъде възможно това обаче, граматиката трябва да има определена форма.

Граматика с такава форма се означава като $LL(1)$. Първата буква "L" означава, че входният поток се сканира отляво-надясно; второто "L" означава прилагане на правилата отляво-надясно; а „1“ означава поглеждане с един символ напред във входния поток.

В ГРАМАТИКА $LL(1)$ НЕ МОЖЕ ДА ИМА ЛЯВО РЕКУРСИВНИ ПРАВИЛА.

Това е необходимо, но не достатъчно условие за $LL(1)$ граматика. Съществуват граматики без ляво рекурсивни правила, които не са $LL(1)$. Също така съществуват граматики, които не могат да бъдат преобразувани в $LL(1)$. В такива случаи трябва да бъдат използвани други методи за синтактичен анализ или да бъдат реализирани специални правила.

Рекурсивно спускане

Първата техника за реализация на синтактичен анализатор с предсказване се нар. *рекурсивно спускане*. Анализатор с рекурсивно спускане се състои от множество малки функции, по една за всеки нетерминален символ в граматиката. При разпознаване на изречение извикваме функциите, съответстващи на нетерминала от лявата страна на правилото, което прилагаме. Ако правилото е рекурсивно, функцията се извиква рекурсивно.

Нека са дадени правила от граматиката на опростен програмен език. В този програмен език всички функции започват с ключова дума FUNC:

program $\rightarrow$ function_list

function_list $\rightarrow$ function_list function | function

function $\rightarrow$ FUNC identifier (parameter_list) statements

- Примерна реализация на C за разпознаване на функция в съответствие с горната граматика има следното действие: очаква първо лексема FUNC, след това идентификатор (име на функцията), последван от отваряща скоба и т.н. Всеки прочетен символ от входния поток се сравнява с очаквания според съответното правило. Ако не съответства, се извежда съобщение за грешка.
- За всеки нетерминален символ от правилото се извиква съответна функция, която го разпознава.

```

void ParseFunction() {
    if (lookahead != T_FUNC) {    // всеки символ, различен от FUNC е грешка
        printf("syntax error \n");
        exit(0);
    }
    else
        lookahead = nexttoken();    // глобална пром. 'lookahead' съдържа
                                    // следващата лексема

    ParseIdentifier();
    if (lookahead != T_LPAREN) {
        printf("syntax error \n");
        exit(0);
    }
    else
        lookahead = nexttoken();
    ParseParameterList();
    if (lookahead != T_RPAREN) {
        printf("syntax error \n");
        exit(0);
    }
    else
        lookahead = nexttoken();
    ParseStatements();
}

```

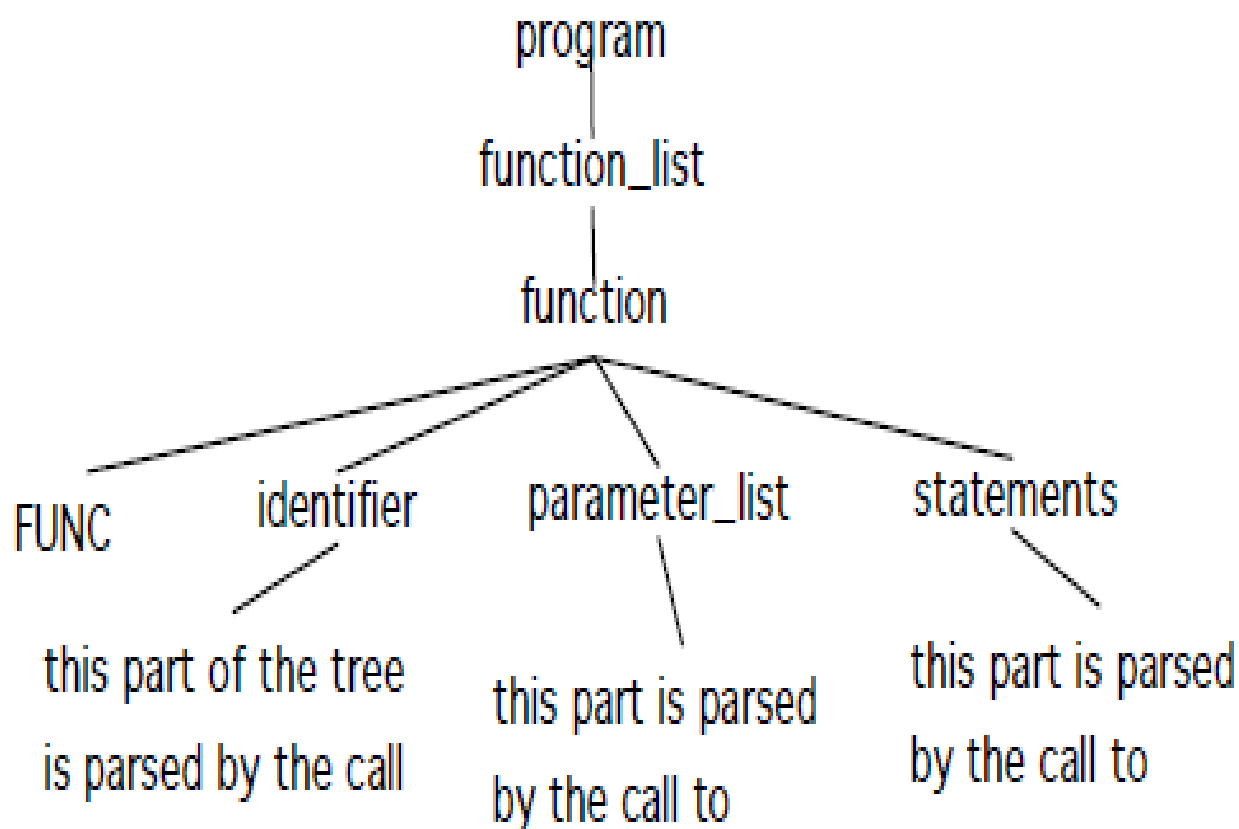

За по-ясно представяне на показаната реализация въвеждаме помощна функция, която проверява дали поредният прочетен символ съвпада с очаквания в правилото и извежда грешка в противен случай. Тази функция ще бъде използвана във всички функции на анализатора.

```
void MatchToken(int expected)
{
    if (lookahead != expected) {
        printf("syntax error, expected %d, got %d\n",
               expected, lookahead);
        exit(0);
    } else // ако съответства, четете следващата лексема
        lookahead = nexttoken();
}
```

С използване на тази допълнителна функция **ParseFunction** изглежда по следния начин:

```
void ParseFunction()
{
    MatchToken(T_FUNC);
    ParseIdentifier();
    MatchToken(T_LPAREN);
    ParseParameterList();
    MatchToken(T_RPAREN);
    ParseStatements();
}
```

Следващата диаграма илюстрира построяване на синтактичното дърво:



Какво се случва, ако имаме множество алтернативни клонове в дясната страна направило? Например:

statement → assg_statement | return_statement | print_statement |
null_statement | if_statement | while_statement | block_of_statements

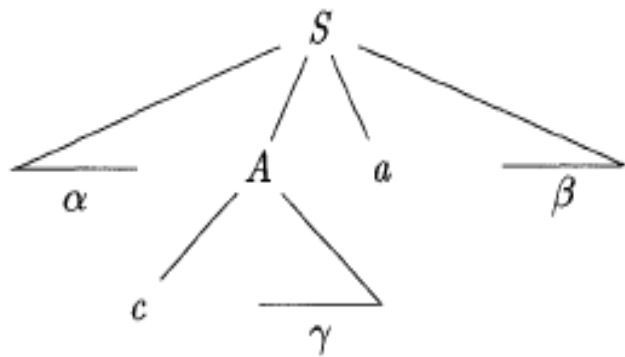
Проблемът при реализацията на функцията **ParseStatement** е коя от всичките седем възможности съответства на определен входен символ. Целта е да определим съответстващия клон от правилото без връщане назад и **поглеждане напред само с един символ**, т.е. изборът на клон от правило трябва да бъде направен с минимална информация.

Дефиниции

При построяване на синтактичен анализатор (независимо дали е отгоре-надолу или отдолу-нагоре) се използват две множества, *First* и *Follow*, свързани с граматиката G . При анализ отгоре-надолу, *First* и *Follow* позволяват да изберем кое следващо правило да приложим на базата на следващия прочетен символ. Тези две множества се използват и при възстановяване след синтактична грешка.

First(α), където α е низ от символи от граматиката, представлява множество от терминални символи, с които започва α . Ако $\alpha \rightarrow^* \epsilon$, то ϵ принадлежи на **First (α)**.

Например, ако $A \rightarrow^* c\gamma$, то c принадлежи на **First (A)**



First може да се използва при синтактичен анализ с предсказване.

Пример:

Дадено е правило за A с два клона: $A \rightarrow \alpha \mid \beta$, където $\text{First}(\alpha)$ и $\text{First}(\beta)$ са непресичащи се множества.

$$\text{First}(\alpha) \cap \text{First}(\beta) = \emptyset \dots\dots\dots (\text{правило 1})$$

Изборът на правило може да се направи чрез следващия символ a , който може да принадлежи само на едно от множествата $\text{First}(\alpha)$ или $\text{First}(\beta)$, но не и на двете.

Например, ако a принадлежи на $\text{First}(\alpha)$, избираме правилото $A \rightarrow \alpha$.

Нека е дадено правило за A с множество алтернативи: $A \rightarrow u_1 \mid u_2 \mid \dots$. За да реализираме функция за синтактичен анализ с рекурсивно спускане, **е необходимо всички множества $\text{First}(u_i)$ да не се пресичат**. Общата форма на такава функция е:

```
void ParseA() {  
    // case below not quite legal C, need to list symbols individually  
    switch (lookahead) {  
        case First(u1): // избор на правило A -> u1  
            /* код за разпознаване на u1 */  
            return;  
        case First(u2): // избор на правилото A -> u2  
            /* код за разпознаване на u2 */  
            Return;  
        ....  
        default:  
            printf("syntax error \n");  
            exit(0);  
    }  
}
```

За изчисляване на $\text{First}(X)$ за всички нетерминални символи X от граматиката, прилагаме следните правила докато нови терминални символи или ϵ могат да бъдат добавени към множеството First .

1. Ако X е терминал, то $\text{First}(X) = \{X\}$.
2. Ако X е нетерминал $X \rightarrow Y_1 Y_2 \dots$, където Y_k е правило за $k \geq 1$, записваме a в $\text{First}(X)$, ако за някоя стойност на i , a принадлежи на $\text{First}(Y_i)$, а ϵ е във всяко от множествата $\text{First}(Y_1) \dots, \text{First}(Y_{i-1})$; т.е., $Y_1 \dots Y_{i-1} \rightarrow \epsilon$. Ако ϵ принадлежи на $\text{First}(Y_j)$ за всяко $j = 1, 2, \dots, k$, тогава добавяме ϵ към $\text{First}(X)$.

Например всеки символ от $\text{First}(Y_1)$ със сигурност принадлежи на $\text{First}(X)$. Ако Y_1 не извежда ϵ , тогава не добавяме нови символи към $\text{First}(X)$, но ако $Y_1 \rightarrow \epsilon$, тогава добавяме $\text{First}(Y_2)$ и т.н.

3. Ако $X \rightarrow \epsilon$ е правило, тогава добавяме ϵ към $\text{First}(X)$.

Изчисляването на First за всеки низ $X_1X_2 \dots X_n$ става по следния начин.

Добавяме към $\text{First}(X_1X_2 \dots X_n)$ всички символи различни от ϵ , които принадлежат на $\text{First}(X_1)$.

Ако ϵ принадлежи на $\text{First}(X_1)$, добавяме символите различни от ϵ , принадлежащи на $\text{First}(X_2)$.

Ако ϵ принадлежи на $\text{First}(X_1)$ и на $\text{First}(X_2)$, добавяме символите различни от ϵ , принадлежащи на $\text{First}(X_3)$.

и т.н.

Накрая добавяме ϵ към $\text{First}(X_1X_2 \dots X_n)$, ако за всяко i ϵ принадлежи на $\text{First}(X_i)$.

Множеството FOLLOW за нетерминалния символ A се определя като множество от терминални символи, които могат да следват отдясно след A във валидно изречение.

За всяко валидно изречение $S \Rightarrow^* uAv$, където v започва с терминален символ, който принадлежи на Follow(A):

$$\text{First}(A) \cap \text{Follow}(A) = \emptyset \dots\dots\dots (\text{Правило 2})$$

Неформално множеството FOLLOW може да се разбира по следния начин: **A** може да се появява на различни места в едно валидно изречение. Множеството FOLLOW описва какви терминали могат да следват низа, който се разширява от **A**. Множеството FOLLOW определя десния контекст на даден нетерминал.

Използвайки тези две дефиниции (FIRST и FOLLOW), можем да обобщим как се обработва правило от вида $A \rightarrow u_1 \mid u_2 \mid \dots$, в синтактичен анализатор с рекурсивно спускане. Във всички случаи трябва да бъде обработен всеки клон от $\text{First}(u_i)$. Ако съществува нетерминал u_i , който извежда ϵ (т.е. празен низ), тогава трябва да бъдат обработени символите от $\text{Follow}(A)$.

```
void ParseA() {  
    switch (lookahead) {  
        case First(u1):  
            /* код за разпознаване на u1 */  
            return;  
        case First(u2):  
            /* код за разпознаване на u2 */  
            return;  
        ...  
        case Follow(A): // избор на правило A->epsilon  
            /* usually do nothing here */  
        default:  
            printf("syntax error \n");  
            exit(0);  
    }  
}
```

Изчисляване на множеството Follow

За всеки нетерминал в граматиката се изпълнява следното:

1. Записваме EOF в $\text{Follow}(S)$, където S е стартовият символ, а EOF е маркер за край на входния поток. Маркерът може да бъде край на файл, нов ред или специален символ, който е очакван за край в използваната граматика. Ще използваме символ \$ за край.
2. За всяко правило $A \rightarrow uBv$, където u и v са низове от символи от граматиката, а B е нетерминал, всички символи от $\text{First}(v)$ без ϵ се записват в $\text{Follow}(B)$.
3. За всяко правило $A \rightarrow uB$ или $A \rightarrow uBv$, където $\text{First}(v)$ съдържа ϵ (т.е. v е празен) всички символи от $\text{Follow}(A)$ се добавят към $\text{Follow}(B)$.

Лява рекурсия

Граматика, която съдържа ляво рекурсивни правила, не е LL(1) граматика.

Пример за ляво рекурсивни правила:

$$A \rightarrow A\alpha$$
$$A \rightarrow \alpha$$

Пример 2: Разглеждаме ляво рекурсивно правило за списък от една или повече функции:

$$\text{function_list} \rightarrow \text{function_list function} \mid \text{function}$$
$$\text{function} \rightarrow \text{FUNC identifier (parameter_list) statement}$$

Реализацията на това правило ще доведе до безкрайна рекурсия!

```
void ParseFunctionList()
{
    ParseFunctionList();
    ParseFunction();
}
```

Премахване на лява рекурсия

Необходимо е да премахнем лявата рекурсия. Тя може да бъде заменена с включване на допълнителни нетерминали или конвертиране в дясна рекурсия, т.е.:

$$A \rightarrow \alpha A$$
$$A \rightarrow \alpha$$

За да реализираме разпознаваща функция за правило `function_list`:

```
function list → function list function | function
```

, заместване :

```
function list -> function function list | function
```

, при което разпознаващата функция добива следния вид:

void ParseFunctionList()

$$\{$$

```
ParseFunction();
```

```
ParseMoreFunctions();    // може да бъде отсъства(т.е.  
                          //разширяваме с празен низ)
```

}

Преобразуване БНФ –
синтактичен – граф – програма
за синтактичен анализ.
Таблично управляем анализ

Много често работата на синтактичния анализатор се представя чрез т.нар. синтактичен граф.

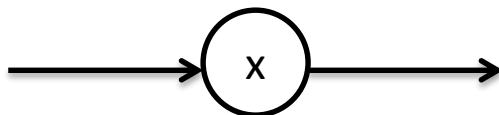
Отличителна черта на анализа отгоре-надолу (top-down) е, че целта на анализа е предварително известна – разпознаване на изречение (символен низ), което може да бъде породено (генерирано) от стартовия символ. Прилагането на дадено правило съответства на разделяне на основната задача на множество от подзадачи.

Програмната реализация съответства на нетерминалните символи и съответните подзадачи.

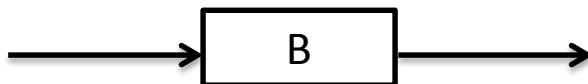
Неоходимо е да построим граф, който представя цялата програма за целите на синтактичния анализ.

Построяване на синтактичен граф

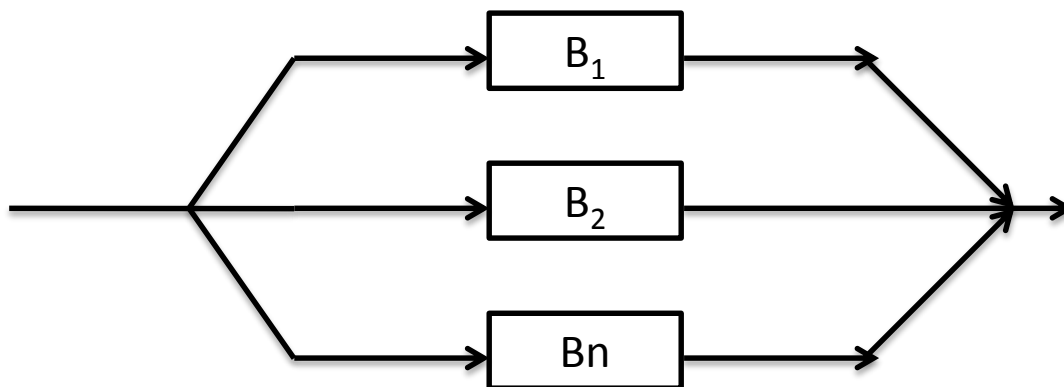
1. Терминален символ



2. Нетерминален символ



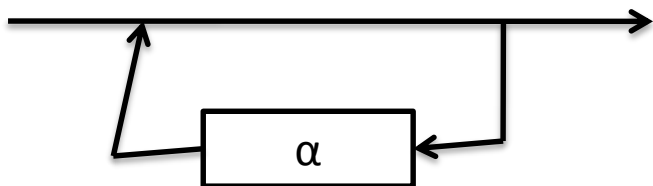
3. Правило от вида $A ::= B_1 \mid B_2 \mid \dots \mid B_n$



4. Правило от вида $A ::= B_1 B_2 \dots B_n$



5. Правило от вида $V ::= \{\alpha\}$



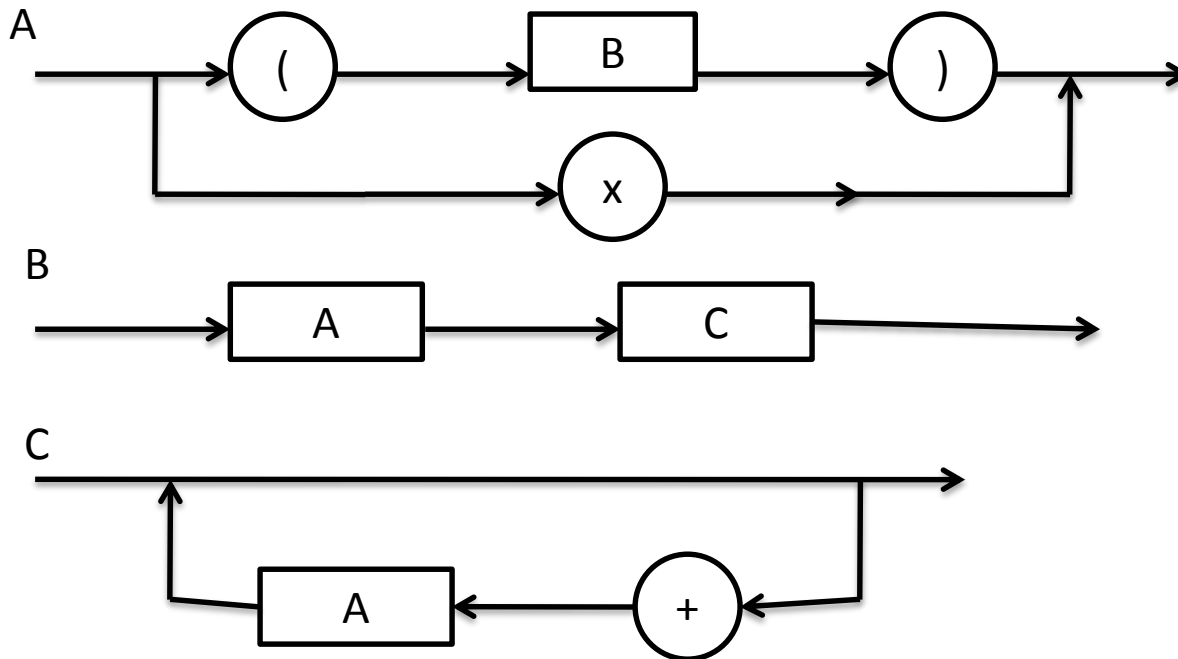
Пример

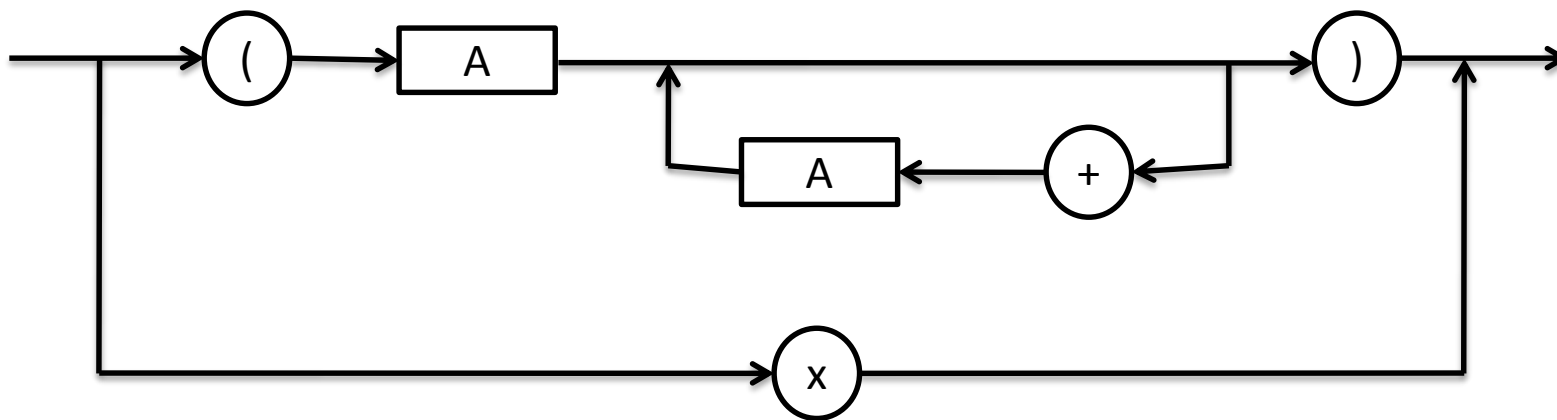
$A ::= x \mid (B)$

$B ::= AC$

$C ::= \{+A\}$

поражда $x, (x), (x+x), ((x))$





Преобразуване на граф в програма



1. Последователно
{
 P(S1);
 P(S2);

 P(Sn);
}

2. Разклонение

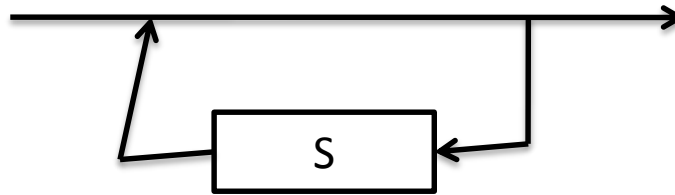
A.

```
if (ch IN L1) P(S1);  
else if ( ch IN L2) P(S2);  
    . . .  
else if (ch IN Ln ) P(Sn);  
else error();
```

B.

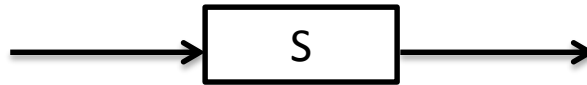
```
switch (ch) {  
    case L1: P(S1); break  
    case L2: P(S2); break;  
    . . .  
    case Ln: P(Sn); break;  
    default: error();; break;  
}
```

Where $L_i = \text{first}(S_i)$



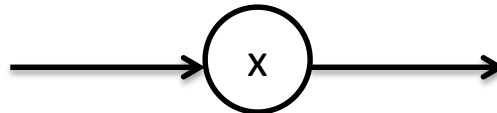
3. Повторение

```
while (ch IN L) { P(S); }
```



4. Нетерминал

```
P(S);
```



5. Терминал

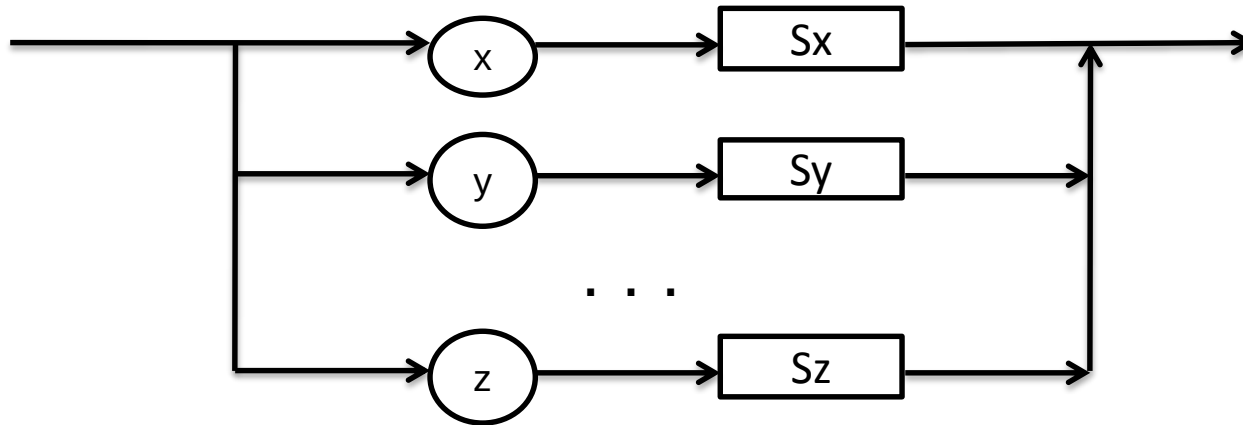
```
if (ch == x)
    ch = getchar ();
else
    error ();
```


Програмна реализация на анализ

```
# include <stdio.h>
extern "C" void error ();
void B();
void C();
char ch;
void A() {
    if (ch == '(' ) {
        ch = getchar ();
        B ();
        if ( ch == ')' ) ch = getchar ();
        else error ();
    } else
        error ();
}
```

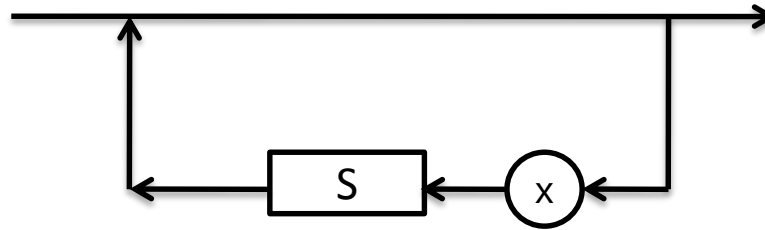
```
void B () {  
    A ();  
    C ();  
}  
void C () {  
    while ( ch == '+' ) {  
        ch = getchar ();  
        A ();  
    }  
}  
void main () {  
    ch = getchar ();  
    A();  
}
```

6. Разклонение с допълнителни нетерминални символи



```
if (ch == 'x' ) {ch = getchar(); P(Sx); }  
else if (ch == 'y') { ch = getchar (); P(Sy); }  
...  
else if (ch == 'z' ) { ch = getchar (); P(Sz);  
else error();
```

Повторение с допълнение



```
while (ch == 'x' ) {  
    ch = getchar ();  
    P(S);  
}
```

8. Заместване на “while” с “do while”

```
ch = getchar();
```

```
P(S);
```

```
while (ch == 'x' ) {
```

```
    ch = getchar ();
```

```
    P(S);
```

```
}
```

```
do {
```

```
    ch = getchar ();
```

```
    P (S);
```

```
} while (ch == 'x');
```

Таблично управляван анализатор

- Построява се таблица T за граматика G
- За всяко правило $A \rightarrow \alpha$ от G :
 - За всеки терминал t от $\text{First}(\alpha)$
 - $T[A, t] = \alpha$
 - Ако ϵ е от $\text{First}(\alpha)$ за всяко t от $\text{Follow}(A)$
 - $T[A, t] = \alpha$
 - Ако ϵ е от $\text{First}(\alpha)$ и $\$$ е от $\text{Follow}(A)$
 - $T[A, \$] = \alpha$

Таблица

$E \rightarrow TX$

$T \rightarrow (E) \mid \text{int}Y$

$X \rightarrow +E \mid \varepsilon$

$Y \rightarrow *T \mid \varepsilon$

$\text{Follow}(X) = \{\$,)\}$

$\text{Follow}(Y) = \{+, \$,)\}$

	(	)	+	*	int	\$
E	TX				TX	
T	(E)				intY	
X	Error	ε	+E			ε
Y		ε	ε	*T		ε

Проблем

- Пример:

$S \rightarrow Sa \mid b$

$\text{First}(S) = \{b\}$

$\text{Follow}(S) = \{\$, a\}$

	a	b	\$
S		b Sa	

Извод

- Ако в таблицата има колона с два записа, то граматиката G не е $LL(1)$, т.е. е в сила едно от следните твърдения:
 - Не е с лява факторизация
 - Има лява рекурсия
 - Нееднозначна
- Граматиките на много програмни езици са контекстно-свободни, но не са $LL(1)$

Синтактичен анализ отдолу- нагоре

Синтактичният анализатор отгоре-надолу започва от стартовия символ. Построяването на синтактичното дърво е в посока от корена на дървото към листата (т.е. отгоре-надолу). Правилата се прилагат до достигане на терминални символи в листата.

Синтактичният анализатор отдолу-нагоре започва с разпознавания низ (от терминални символи). Построяването на синтактичното дърво е в посока от листата към корена (т.е. отдолу-нагоре). Правилата се прилагат в обратен ред. Синтактичният анализатор търси в разпознавания низ такъв подниз, който съответства на дясната страна на някое от правилата. Ако намери такъв подниз, анализаторът го замества с нетерминалния символ в лявата част на съответното правило, т.е. разпознаваният низ се редуцира. Целта е разпознаваният низ да бъде редуциран само до стартов символ. Това означава, че входният низ е успешно разпознат.

В общия случай синтактичният анализ отдолу-нагоре е по-мощен в сравнение с анализа отгоре-надолу, но е по-труден за реализация.

Синтактичен анализатор отдолу-нагоре използва *стек* за съхраняване на текущата позиция и *таблица* за определяне на следващата операция.

Алгоритъмът на синтактичния анализ отдолу-нагоре чете лексеми от входния поток и ги въвежда в стека, опитвайки се да построи последователности, които се срещат в дясната част на някое от правилата на граматиката. При откриване на такава последователност тя се замества в стека със съответния нетерминал от лявата страна на правилото. Построяването на синтактичното дърво продължава във възходяща посока, обратна на посоката при анализ отгоре-надолу. При успешно разпознаване би трябвало всички лексеми от входния поток да бъдат въведени в стека и евентуално да се получи последователност, съответстваща на дясната част в правилото на стартовия символ на граматиката.

Пример:

Дадени са правила на граматика:

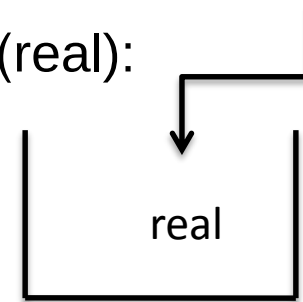
1. $\langle S \rangle \rightarrow \text{real } \langle \text{IDLIST} \rangle$
2. $\langle \text{IDLIST} \rangle \rightarrow \langle \text{IDLIST} \rangle , \langle \text{ID} \rangle$
3. $\langle \text{IDLIST} \rangle \rightarrow \langle \text{ID} \rangle$
4. $\langle \text{ID} \rangle \rightarrow A|B|C|D$

Да се получи следното изречение с използване на метод отдолу-нагоре:

real A, B, C

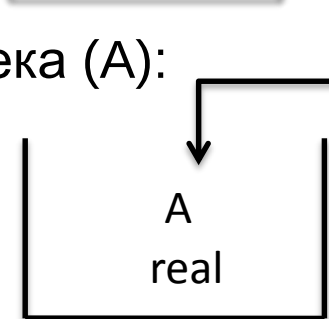
Стъпка 1. Първата лексема се записва в стека (real):

real A, B, C
↑

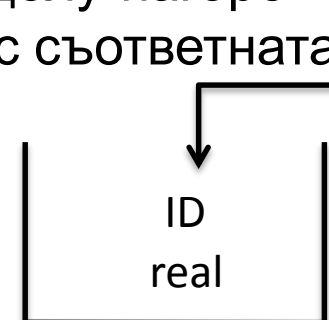


Стъпка 2. Следващата лексема се записва в стека (A):

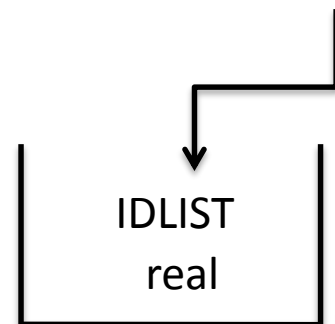
real A, B, C
↑



Стъпка 3. Замества се A според правило 4. Тази операция се нарича *редуциране на стека*. При анализ отдолу-нагоре заместваме дясната страна на правилата със съответната лява страна.



Стъпка 4. Замествахме в стека според правило 3



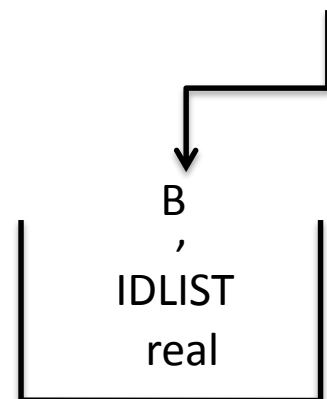
Стъпка 5. Записваме следващата лексема (,)

real A , B, C

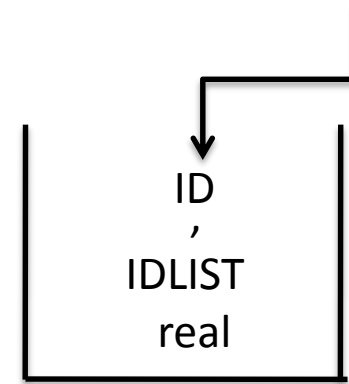


Стъпка 6. Записваме следващата лексема (B)

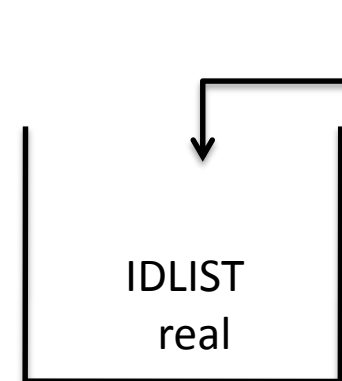
real A , B , C



Стъпка 7. Редуцираме стека според правило 4.



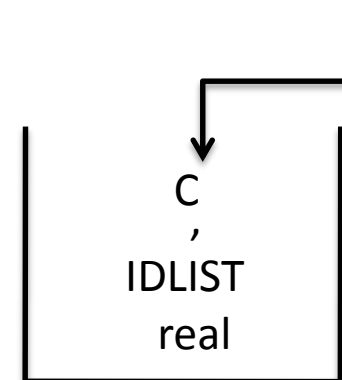
Стъпка 8. Редуцираме стека според правило 2.



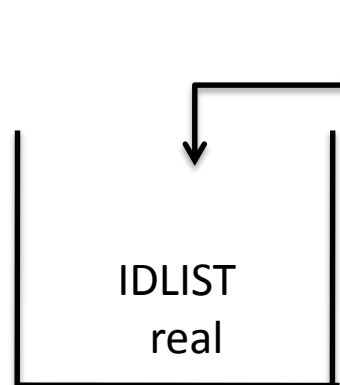
Стъпка 9. Записваме следващата лексема (C)

Стъпка 10. Четем следващата лексема

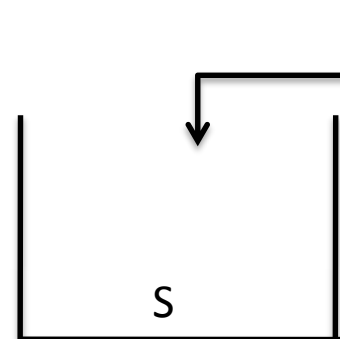
real A , B , C ↑



Стъпка 11. Редуцираме стека според правила 4 и 2



Стъпка 12. Редуцираме стека според правило 1



- Описаният анализатор се нар. *анализатор с изместване и редуциране*. На всяка стъпка такъв анализатор изпълнява една от следните три операции:

Изместване: Ако не е възможно да се изпълни редуциране на стека и има останали лексеми във входния поток, тогава прехвърляме лексема от потока в стека. Тази операция се нар. *изместване*. Например, нека за горната граматика разгледаме стек със съдържание (**real IDLIST** ,) и входен поток (**B,C**). Невъзможно е да се изпълни редуциране на стека, защото съдържанието му не съответства на дясната страна на никое от правилата. В този случай записваме първата лексема от потока в стека (в случая в стека се записва B, а C остава във входния поток).

Редуциране: Ако съществува правило $A \rightarrow w$ и съдържанието на стека е (**qw**) за низ q (q може да е празен), тогава съкращаваме (редуцираме) стека до (**qA**). Правилото за нетерминала A се прилага отдясно-наляво. Например, за горната граматика, ако съдържанието на стека е (**real ID**), можем да използваме правилото $IDLIST \rightarrow ID$, за да редуцираме стека до (**real IDLIST**).

Ако при редуцирането в стека се съдържа стартовият символ и няма останали лексеми във входния поток, то входният низ се разпознава като валидно изречение. Това е последната стъпка на успешен анализ.

Грешка: Ако символите, записани в стека, не съответстват на дясната страна на никое от правилата, не може да се приложи редуциране на стека. Също, ако записването на следваща лексема в стека води до последователност, която не може да бъде редуцирана до стартовия символ, няма смисъл да се прилага редуциране. Описаните случаи означават, че входният поток не е валидно изречение.

Проблем

Описаните типове граматики създават конфликти, които се нар. *изместване-редуциране* и *редуциране-редуциране*. Първият вид конфликт се среща, когато анализаторът не може да реши дали да измества или да редуцира. Вторият вид конфликти се среща, когато анализаторът има повече от едно правило за редукция.

Пример за конфликт изместване-редуциране възниква при конструкцията *if-then-else*.

Типично правилото за тази конструкция е:

$$S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S$$

Разглеждаме какво ще се получи при анализатор с изместване и редуциране:

$$\text{if } E \text{ then if } E \text{ then } S \text{ else } S$$

В даден момент от анализа стекът ще има следното състояние:

$$\text{if } E \text{ then if } E \text{ then } S$$

, при което *else* ще бъде следващата лексема.

Възможно е да се извърши редуциране на разпознавания низ, защото съдържанието на стека съответства на дясната страна на първото правило. Възможно е и изместване на *else* за построяване на дясната страна на второто правило. Редуцирането затваря вътрешният оператор *if*, като така свързва *else* с външния *if*. Изместването свързва вътрешния *if* с *else*.

Проблемът в примера е как анализаторът да избере операция – изместване или редуциране.

И двете операции са валидни спрямо граматиката, но получените като резултат синтактични дървета са различни.

Проблемът се свежда до избор на оператор *if*, към който принадлежи *else* – към вътрешен или към външен *if*.

В C и Java оператор *else* се отнася към най-близкия оператор *if*. При други езици като Ada и Modula този проблем се решава чрез затварящ оператор *endif*.

Проблем *редуциране-редуциране* се среща рядко и обикновено е следствие от проблем в дефинирането на граматиката.

За вземане на решение каква операция да бъде приложена се използват *детерминирани методи*.

Например, ако се използва терминиращ символ (;), тогава при срещане на този символ анализаторът трябва да редуцира, в противен случай да измества. Например в горната граматика за съдържание на стека “**real IDLIST**“, ако следващият символ е “;”, ще се изпълни редуциране, а ако следващият символ е “,”, ще се изпълни изместване.

Имайки обща представа как работи анализатор с изместване и редуциране, ще разгледаме как обработва входен низ и как определя кое правило да приложи при редуциране. За целта ще разгледаме метод за реализация нар. LR анализ. Този метод е приложим за всички детерминирани езици и граматики.

LR анализ

LR анализаторите ("L" означава, че входният низ се преглежда отляво-надясно, "R" означава дясно извеждане на изречение) са ефективни, таблично управляеми анализатори с изместване и редуциране. Класовете граматики, които могат да бъдат породени с LR метод, са по-голямо множество от класовете граматики, които са породени чрез LL. Всички програмни конструкции, които се дефинират с контекстно-свободни граматики, могат да бъдат реализирани чрез LR метод. Допълнително предимство на LR е, че при повечето граматики няма необходимост от преработване на правилата, както е при LL граматиките.

LR анализаторът използва две таблици:

1. *Таблица на операциите $Action[s,a]$* , която указва на анализатора каква операция да изпълни при състояние s във върха на стека и при следващ терминален символ a в разпознавания низ. Възможните операции са изместване на състояние в стека, редуциране на символ във върха на стека, приемане на низ (разпознаване на изречение) или маркиране на грешка.
2. *Таблица $Goto[s,X]$* показва кое да е новото състояние във върха на стека след редуциране на нетерминалния символ X при състояние s във върха на стека.

Двете таблици се използват комбинирано, като таблицата на операциите съдържа редове за терминални символи, а *Goto* таблицата съдържа редове за нетерминалните символи. Обединената таблица се нарича *LR парсираща таблица*.

Пример за LR парсираща таблица

Състояние	S	IDLIST	ID	real	,	A B C D	;
1	STOP HALT			S2			
2		S5	S4			S3	
3					R4		R4
4					R3		R3
5					S6		R1
6			S4			S3	
7					R2		R2

Редовете в LR таблицата съответстват на състоянията, в които може да се намира анализаторът.

Всяко състояние съответства на позиция от граматическо правило, което е достигнато от анализатора.

Анализаторът има 2 стека – стек за символи и стек за състояния.

Таблицата се състои от 4 типа елементи:

1. Изместващи елементи. Те имат форма Sx . Изместващ елемент има следното значение:
 - Да постави в символния стек символа, съответстващ на текущата колона.
 - Да постави в стека на състоянията състоянието x и да направи преход в състояние x .
 - Ако следващият входен символ е терминален, да го прочете

2. Изместващи елементи с форма Ry . Такъв елемент има следното значение:
 - Редуцира с правило y .
 - Ако дясната страна на правилото y има n символа, тогава извлича (pop) n символа от символния стек и от стека на състоянията.
 - Преход към състоянието във върха на стека на състоянията.
 - Нетерминалният символ в лявата страна на правило y се приема за следващия входен символ.
3. Елементи за грешка. Всяка празна позиция в таблицата съответства на синтактична грешка.
4. Елемент за край (един или повече). Тези елементи приключват анализа.

Пример

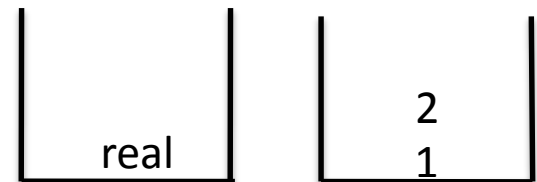
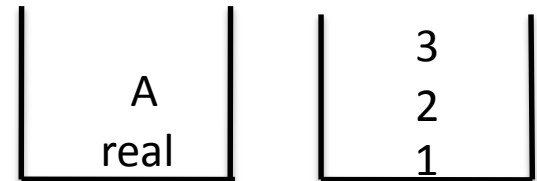
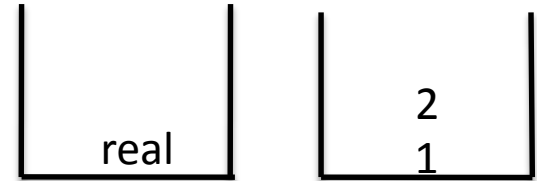
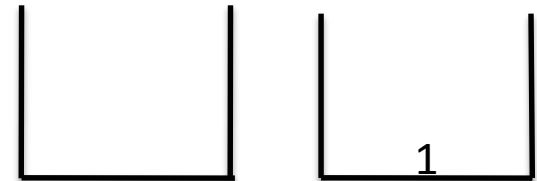
Пораждане на изречение
real A,B,C;

Начално състояние S1
↑
real A,B,C;

Входен символ real – S2: изместване в 2,
входен символ става A
↑
real A,B,C

Входен символ A – S3:
↑
real A,B,C

Входен символ , - R4: замества правило 4



Пример

Входен символ ID, преход в състояние 4
real A,B,C;



ID
real

4
2
1

Входен символ , – R3: замества правило 3,
real A,B,C



real

2
1

Входен символ IDLIST – S5: измества в съст. 5:
real A,B,C



IDLIST
real

5
2
1

Входен символ , - S6: изместване в състояние 6
real A,B,C



,
IDLIST
real

6
5
2
1

Семантичен анализ

Синтактичният анализ проверява дали програмата се състои от лексеми, подредени в синтактично коректна последователност.

Семантичният анализ изследва смисъла на входната програма (разпознато изречение). Семантиката се явява изображение от *програма* на езика към *модел* на нейното поведение.

Примери за необходими условия при семантично коректна програма:

- всички променливи, функции, класове да бъдат подходящо дефинирани и да бъдат използвани в съответната област на видимост;
- променливи и изрази трябва да бъдат използвани с подходящи типове данни;
- контрол на достъпа (напр. до методи на клас).

На етапа на семантичния анализ приключва анализа (т.е. проверката за коректност) на програмата и започва генериране на код.

Основни цели на семантичния анализ:

Проверка на правилата за използване на всички конструкции, идентификатори (променливи, функции и др.) и константи на базата на:

- Правила за видимост в езика (блоковата структура на езика);
- Съвместимост на типове
- Ограничения на конкретната реализация
- Допълнителни изисквания

Важна част от семантичния анализ е обработване на декларациите на променливи/функции/типове и проверка на типове данни. В много езици идентификаторите трябва да бъдат декларирани преди да бъдат използвани. Когато семантичният анализатор срещне нова декларация (напр. на променлива или функция от някакъв тип данни), той записва информация в символната таблица за типа на данните. Когато в останалата част от програмата анализаторът срещне същия идентификатор, той проверява дали типът данни на идентификатора позволява да бъдат изпълнени съответните операции.

Примери за проверки, извършвани на етапа на семантичния анализ:

- Типът на данните в дясната страна на оператор за присвояване трябва да съвпада с типа на данните в лявата страна, а идентификаторът в лявата страна трябва да има подходяща декларация.
- Формалните параметри на функция трябва да съответстват по брой и по тип на параметрите, с които е извикана функцията (фактически параметри).
- Някои езици изискват имената на идентификаторите да са уникални. Следователно семантичният анализатор трябва да извежда грешка при срещане на два глобално деклариран идентификатора с еднакви имена.
- Операндите в аритметичните изрази трябва да са от числов тип – някои езици изискват дори еднакъв тип на операндите (т.е. напр. без int-to-double преобразуване)

Типове и декларации

Тип данни представлява множество от стойности и операции върху тях. В повечето програмни езици съществуват три групи типове:

- Базови типове **int, float, double, char, bool, etc.** Това са основни типове данни, които се предоставят във всички езици. Може да има възможност за дефиниране на потребителски тип данни на базата на основните типове (напр. С **enums**).
- Съставни типове **arrays, pointers, records, structs, unions, classes** и др. These types are constructed as aggregations of the base types and simple compound types.
- Динамични типове данни **lists, stacks, queues, trees, heaps, tables**

В много езици първо трябва да бъдат декларирани име и тип на данните (на променлива, функция, тип и др.). Освен това декларациите определят област на видимост. *Декларацията* представлява оператор, който изпраща съответната информация към компилатора. Основно декларацията се състои от име и тип, но в много езици тя включва модификатори за определяне на видимост (напр. **static** в C, **private** в Java). Някои езици позволяват също инициализиране на променливи, при което с един оператор се извършва декларация и задаване на стойност. Пример за декларации в C програма:

```
double calculate(int a, double b);    // function prototype
int x = 0;                            // global variables available throughout
double y;                             // the program
int main()
{
int m[3];                             // local variables available only in main
char *n;
...
}
```

Проверка на типове

Проверката на типовете е процес на сверяване дали всяка операция, зададена в програмата, отговаря на изискванията на езика. Това изисква операндите във всеки израз да бъдат подходящ брой и от подходящ тип.

Голяма част от семантичния анализ се състои в проверка на типовете. Понякога правилата за операциите са дефинирани в части от кода (напр. прототипи на функции), а понякога тези правила са част от дефинициите на езика (напр. правилото „и двата операнда в бинарен аритметична операция трябва да са от един и същ тип“). При откриване на нарушено правило, напр, опит за събиране на стойност на символ със стойност на реално число, анализаторът извежда съобщение за *несъвместимост на типове*.

Даден програмен език е *силно типизиран*, ако всяка грешка, свързана с типовете данни, се открива по време на компилиране.

Проверките с типове данни могат да бъдат направени по време на компилиране (*статична проверка*), по време на изпълнение (*динамична проверка*) или да бъдат разделени между тези два етапа.

Статичната проверка на типове се извършва по време на компилация. Необходимата информация се получава от декларациите в програмата и се съхранява в символната таблица.

Недостатък:

Не всички грешки, свързани с типовете, могат да бъдат открити по време на компилиране. Например, ако на две променливи **a** и **b** от тип **int** са присвоени големи стойности, тяхното произведение **a * b** може да надхвърли диапазона на представимите цели числа или частното на две цели числа може да породи грешка деление на нула. Такива грешки не могат да бъдат открити по време на компилиране.

Динамичната проверка на типове данни се извършва по време на изпълнение.

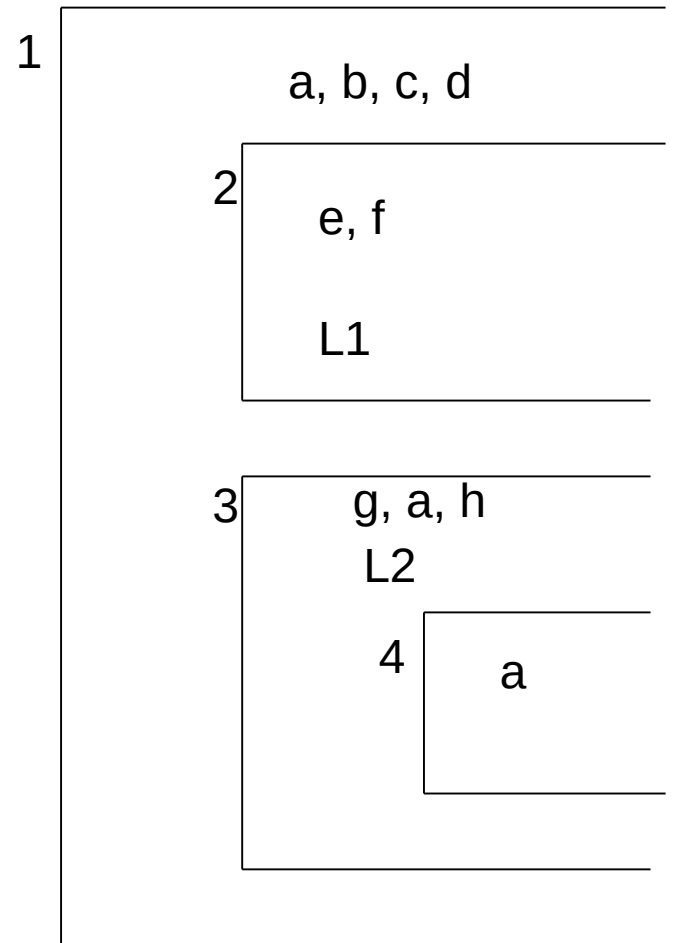
Например запис в символната таблица за променлива от тип *double* ще съдържа освен стойността на променливата и информация за типа (*double*). Изпълнението на всяка операция с тази променлива първо ще проверява информацията за типа на променливата. Операцията ще се изпълнява само, ако типът на променливата позволява.

Например при извикване на операция за събиране първо се проверява дали типовете на операндите са съвместими. LISP е пример за език, при който проверката на типовете се изпълнява динамично. В програма на LISP не е задължително да се задава тип на данните при деклариране на променлива. Поради това компилаторът не може да изпълни никакъв анализ на използваните типове данни. Проверката се извършва по време на изпълнение.

Динамичната проверка на типове прави изпълнението по-бавно, но открива грешки, които не могат да бъдат открити при компилиране.

Блокова структура на програма и област на видимост на променливи

```
Program BLOCK;  
  var a, b, c, d : ...  
  procedure P1;  
    var e, f : ...  
    ...  
  begin ... L1: ... end;  
  procedure P2;  
    var g, a, h : ...  
    procedure P3;  
      var a: ...  
    begin ... end;  
  begin ... L2: ... end;  
begin ... end;
```



Последователност на търсене на идентификатор в блоковете

1. Търсене в текущия блок:
 - Ако идентификаторът е открит, премини към 3.
 - Ако идентификаторът не е открит, премини към 2.
2. Търсене в обхващащия блок:
 - Ако идентификаторът е открит, премини към 3.
 - Ако идентификаторът не е открит:
 - Ако блокът е най-външен, премини към 4
 - Ако блокът не е най-външен, премини към 2
3. Действия в зависимост от семантиката на езика
4. Грешка – недеклариран идентификатор

Таблицы с идентификатори

Идентификаторите се съхраняват в таблици.

Необходима е допълнителна таблица с указатели към списъка с идентификатори за всеки блок.

	i	S	N	P		
Блок	1	0	4		→	a, b, c, d
P1	2	1	3		→	e, f, L1
P2	3	1	4		→	g, a, h, L2
P3	4	3	1		→	a

i – номер на блок

S – номер на обхващащ блок

N – брой идентификатори

P – указател към началото на списъка с идентификатори за блока

$S[N]$ - символна таблица с N елемента;

$B[M]$ – списък от блокове, зададени чрез структури с полета S , N , P

$curr_bl$ – номер на текущия блок

$last_bl$ – номер на най-външния блок (първоначално 0)

top_el - индекс на елемента във върха на стека (първоначално $N + 1$)

$last_el$ – индекс на последния елемент в стека (първоначално 0)

При обработване на блок се изпълняват две процедури – едната се извиква в началото на блок (*open_block()*), а втората се извиква при изход от блок (*close_block()*).

$\langle block \rangle ::= \text{procedure } \langle body \rangle \text{ end}$

начало на блок затваряне на блока

// semantic analysis

const N = 1000;

const M = 100;

char * S[N];

typedef struct {

int S;

int N;

int P;

} block;

block B[M];

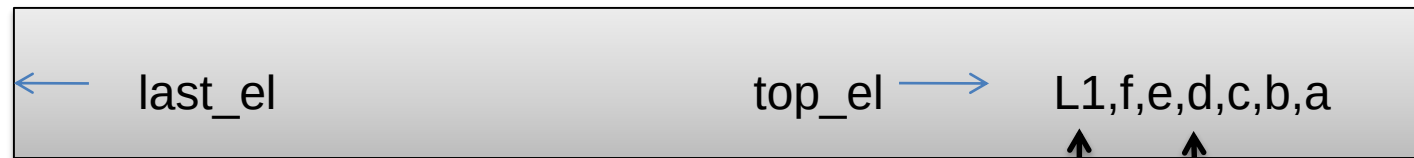
```

void open_block () {
    last_bl = last_bl + 1;
    B[last_bl].S = curr_bl;
    B[last_bl].N = 0;
    B[last_bl].P = top_el;
    curr_bl = last_bl;
}

void close_block () {
    int l;
    B[curr_bl].P = last_el + 1;
    for (i=1; B[curr_bl].N; i++) {
        last_el = last_el + 1;
        S[last_el] = S[top_e];
        top-el = top_el = 1;
    }
    curr_el = B[curr_bl].S;
}

```

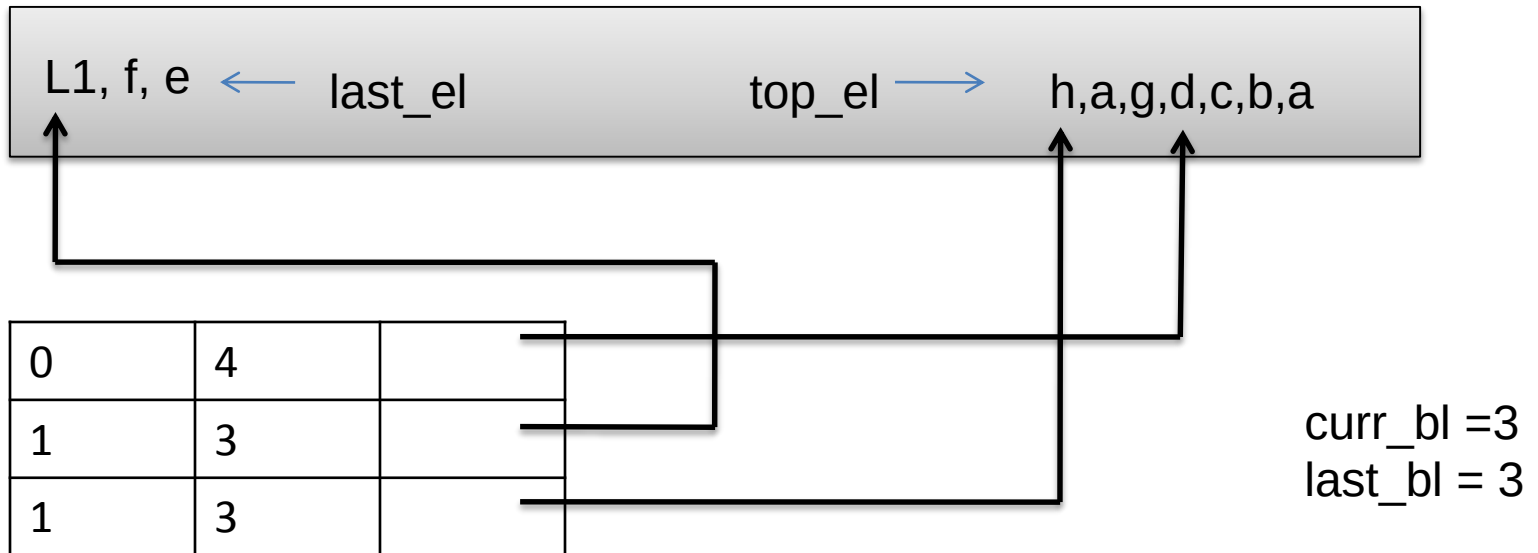
L1



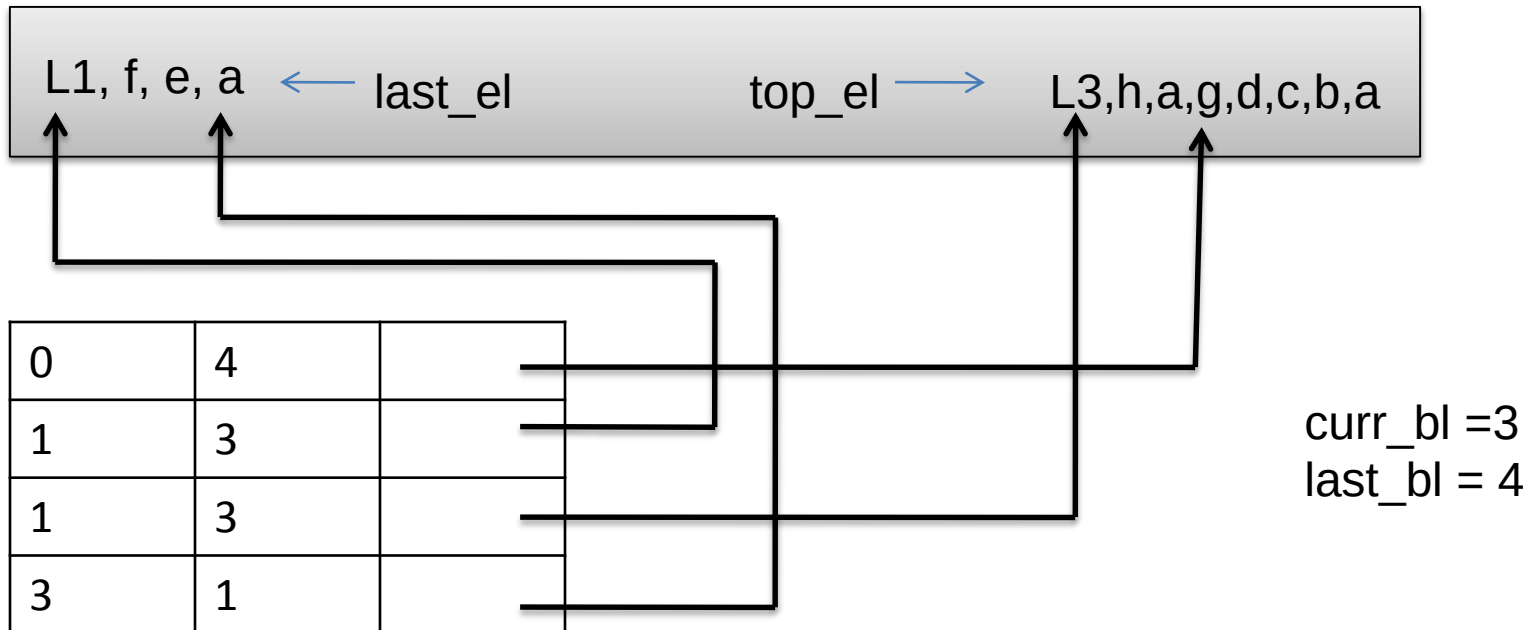
0	4	
1	3	

curr_bl = 2
last_bl = 2

h



L3



Разпределение на паметта при генериране на код

Разпределение на паметта



Дефиниции

При генериране на код се извършва разпределение на паметта и създаване на структури за управление на стек (stack) и heap.

Област за данни – последователност от клетки в паметта, заделени за логически свързани данни.

Понякога (но не винаги) една област от данни съответства на един блок от програмата.

По време на компилирането клетките, заделени за някои променливи могат да бъдат представени чрез двойки от числа (напр. (3,0), (3,1)):

- Първото число означава брой на данните;
- Второто число означава отместване.

При генериране на инструкция тази двойка се конвертира в действителен адрес на променливата. Адресът се получава чрез задаване на базов адрес и регистър на отместване.

Пример:

(брой данни, отместване) => (базов адрес, отместване)

Областта за данни може да бъде:

- статична – паметта се разпределя за цялото време на изпълнение на програмата. Достъпът става чрез абсолютни адреси, а не с двойка `<base, displacement>`.
- динамична – паметта се разпределя динамично по време на изпълнение. Областта от данни може да се създава и изтрива. При изтриване на област всички данни в нея се губят.

Пример:

Процедури за създаване и изтриване на блок от паметта:

- `GETSTORAGE (Base, SIZE)` – създава памет с размер `SIZE` със стартов адрес `Base`. Следващото извикване на `GETSTORAGE` ще резервира друга област от данни (с друг базов адрес). Базовият адрес на създадената памет може да бъде записан в клетка от паметта или в регистър.
- `FREESTORAGE (ADDRESS, SIZE)` – унищожава памет със стартов адрес `ADDRESS` и размер `SIZE`.

Стекова памет (стек)

- Използва се за съхраняване на параметри на функции и на техните локални променливи
- Стековата памет се разширява и свива при запис и четене на локални променливи
- Няма необходимост от управление на паметта от страна на програмиста, защото стойностите на променливите се записват в стека и се изтриват от него автоматично
- Управлява се от CPU – бързо четене и запис
- Стековата памет има ограничен размер
- Променливите съществуват в стека само докато се изпълнява функцията, която ги е създадала

Неар памет

- Управлява се от програмиста (запис и изтриване на променливи) – напр. чрез `malloc()`, `calloc()`, `realloc()`, `free()`
- Данните са достъпни от всички подпрограми (функции и процедури) в програмата
- Няма ограничения за размера на паметта
- По-бавен достъп за четене и запис
- Може да се получи фрагментиране на области от паметта

Представяне на данни

Прости променливи: обикновено се представят в памет с фиксиран размер:

- символи: 1 или 2 байта
- цели числа: 2, 4 или 8 байта
- реални числа: от 4 до 16 bytes
- булеви данни: 1 бит (най-често се използва 1 пълен байт)

Указатели: обикновено се представят като цели числа без знак. Размерът на указателя зависи от адресното пространство на използваната архитектура. Най-често се използват 4 байта за съхраняване на адрес, като по такъв начин се адресира пространство от 4GB.

Представяне на данни

Едномерни масиви: представят се като съседни блокове от елементи. Размерът на масива е равен най-малко на броя на елементите, умножен по размера памет за представяне на един елемент. Елементите се разполагат последователно, започвайки от първия елемент, от малките към големите адреси в паметта. Ако базовият адрес на масива е известен, компилаторът може да генерира код за изчисляване на адреса на всеки елемент чрез отместване на базовия адрес:

$\&arr[i] = arr + 4 * i$ (4-byte integers; $arr = \text{base address}$).

Многомерни масиви: за представяне на многомерни масиви се използва един от следните методи: действителен многомерен масив или масив от едномерни масиви.

Действителен многомерен масив се разполага последователно. Може да се разположи по редове (напр. в C, Pascal), където редовете се разполагат един след друг. Може да се разположи и по колони (напр. Fortran), където колоните се разполагат една след друга. За масив от цели числа, съставен от m реда и n колони, адресът на всеки елемент се изчислява както следва:

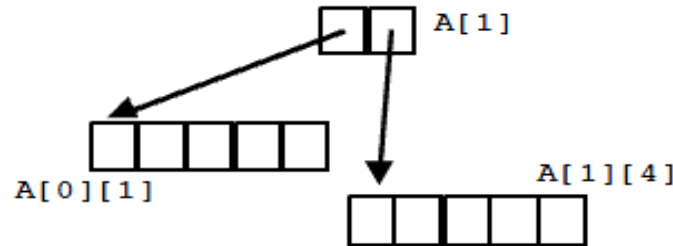
$$\&arr[i,j] = arr + 4 * (n * i + j)$$

Или в по-общ вид за произволен масив:

$array[L1..U1, L2..U2]$ (в Pascal)

$$\&arr[i, j] = arr + (sizeof T) * ((i - L1) * (U2 - L2 + 1) + (j - L2))$$

Алтернативният метод е *масив от масиви*, където елементите на масива са указатели към всеки ред на масива. Например така изглежда масив $A[2][5]$ (първият елемент е с индекс 0):



За прочитане на стойността на елемент $A[i][j]$ е необходимо прочитане на два указателя за разлика от многомерния масив, който изисква прочитане само на един указател. Адресът на ред i се получава чрез следното изчисляване:

$$A + i * \text{sizeof}(\text{pointer}).$$

След прочитане на адреса към него се добавя

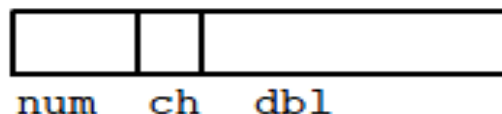
$$j * \text{sizeof}(\text{arrayElem})$$

, за да се получи адрес на елемента, от който след това се получава стойността на елемента.

Масив от масиви изисква повече памет поради използване на допълнителни указатели, докато при многомерните масиви е необходимо пространство само за съхранение на елементите.

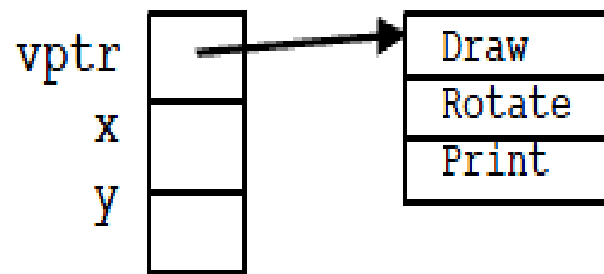
Структури: полетата на структурата се разполагат последователно в непрекъснат блок от малките към по-големите адреси в паметта. Размерът на един запис от структурата е равен най-малко на сумата от размерите на полетата. В дадения пример, ако целите числа заемат памет от 4 байта, символите 1 байт, а реалните числа с двойна точност (тип double) 8 байта, то цялата структура заема 13 байта. Отделните полета са разположени с отмествания съответно 0, 4 и 5 байта в адресното пространство. В много архитектури, обаче, за структурата ще бъдат необходими 16 байта, защото отместванията ще бъдат формирани така, че всяко поле да започва с изравнена граница.

```
struct example {  
    int num;  
    char ch;  
    double dbl;  
};
```



Обекти: разполагат се в паметта подобно на структурите, като полетата са инстанции на променливите на класа. Методите не се съхраняват в самия обект. За динамично разпределяне (dispatch) във всеки обект се използва допълнителен скрит указател, който сочи към памет със споделена информация за методите на класа (често наричана *vtable*). Ако езикът не поддържа динамично разпределяне, ако в класа не са дефинирани методи, които го изискват или ако в класа има само `private` и `final` методи, тогава не е необходима таблица и скрит указател в представянето на обекта.

```
public class Rectangle {  
    private int x, y;  
    public void Draw() {}  
    public void Rotate(int deg) {}  
    public void Print() {}  
}
```



Инструкции: кодират се като битови последователности, обикновено с размер дума. Различните битове в инструкцията кодират информация за типа на инструкцията (четене, запис, умножение и др.), за използваните регистри и др.

При много компилируеми езици не се записва никаква информация за типа на съхраняваните променливи. Така например при прочитане на съдържанието на адрес 1000 не е ясно дали то трябва да се интерпретира като тип данни, указател, инструкция или др. Езици като LISP и Smalltalk поддържат информация за типа данни на променливите по време на изпълнение. При тях всяка променлива се маркира с таг за типа на променливата.

Област на видимост и време на живот

Област на видимост на променлива (scope) определя части от програмата, в които променливата може да бъде достъпвана.
Времето на живот на променливата (lifetime) определя периода от време, в който може да се използва променливата.

глобални: Времето на живот съвпада с времето на живот за цялата програма и областта на видимост също е в цялата програма.

статични: Времето на живот съвпада с времето на живот за цялата програма, но областта на видимост е само във функцията (или модула), в който е декларирана променливата.

локални: (наричани също автоматични) Съществуват само за времето на извикване на подпрограмата, в която са декларирани; нова променлива се създава при влизане в процедура и се унищожава при излизане от подпрограмата. Достъп до локални променливи може да се извършва само в подпрограмата, в която са декларирани.

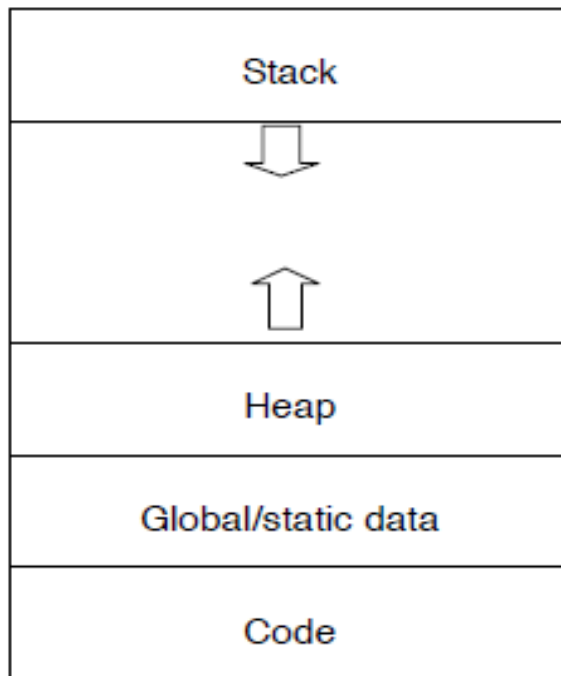
динамични: променливи, които се създават по време на изпълнение на програмата; обикновено се представят чрез адрес, съхраняван в променлива от един от изброените по-горе класове. Времето на живот съвпада с това на програмата, а областта на видимост съвпада с областта на променливата, съхраняваща адреса.

Глобални и статични променливи имат единствена инстанция, която съществува по време на живота на програмата; различават се само в областта на видимост. Обикновено тези два класа променливи се групират заедно в сегмент с глобални/статични данни в изпълнимата програма.

Локалните променливи се създават само при влизане в подпрограма и съществуват до нейното приключване. За тяхното обработване по време на изпълнение се използва стекова памет, в която се съхраняват стойностите на локалните променливи. Областта от паметта, използвана за съхраняване на всички локални променливи в подпрограма, се нар. *стеков фрагмент*. Стековият фрагмент за текущо изпълняваната подпрограма се намира във върха на стека. Адресът на текущия стеков фрагмент обикновено е записан в регистър на микропроцесора и се нар. базов указател.

Динамичното заемане на памет по време на изпълнение обикновено се реализира чрез извикване на библиотечни функции (напр. функция `malloc`). Това адресно пространство се заделя в сегмент от паметта, различен от програмния код, глобални/статични данни и стек. Тази памет се нар. *heap*.

Адресното пространство на изпълняваща се програма се разделя по следния начин:



Стеков фрагмент

При всяко извикване на подпрограма (функция или процедура) за нея се създава стеков фрагмент. В един стеков фрагмент (нар. activation record) се съдържа следната информация:

- 1) *Стеков указател (frame pointer)*: указател към предишния стеков фрагмент. Използва се за връщане към извикваща подпрограма при завършване на текущо изпълняваната подпрограма. Понякога се нарича още динамична връзка.
- 2) *Статична връзка (static link)*: съдържа указател към стековия фрагмент на подпрограмата, в която е декларирана текущата подпрограма.
- 3) *Адрес на връщане*: указател към точката на връщане в кода след приключване на текущата подпрограма.
- 4) *Стойности на параметрите*, с които е извикана подпрограмата и локални променливи.

При извикване на подпрограма (напр. на функция) се изпълняват следните основни стъпки:

Преди извикване на функция извикващата функция прави следното:

- 1) Съхранява стойностите на всички необходими регистри
- 2) Записва в стека стойностите на параметрите на извикваната функция
- 3) Задава статична връзка (ако такава съществува)
- 4) Записва в стека адреса на връщане
- 5) Прави преход към извикваната функция

По време на извикването на функция извикващата функция изпълнява следните действия:

- 1) Съхранява стойностите на необходимите регистри
- 2) Установява новия стеков указател
- 3) Заделя пространство за всички локални променливи
- 4) Изпълнява операторите в извиканата функция
- 6) Възстановява стойностите на всички съхранени регистри
- 7) Прави преход към съхранения адрес на връщане

След извикване на функция извикващата функция изпълнява следните действия:

- 1) Изтрива от стека адреса на връщане и параметрите
- 2) Възстановява стойностите на всички съхранени регистри
- 3) Продължава изпълнението си

Предаване на параметри

Общите методи за предаване на параметри в програмните езици са:

Предаване по стойност: В извиканата подпрограма се копират стойностите на параметрите. По този начин подпрограмата работи с копия на параметрите, направените промени не се отразяват в извикващата подпрограма.

Предаване по референция (адрес): Вместо копие на параметър извикваната подпрограма получава референция към параметъра. Обикновено референцията се реализира чрез указател към параметъра. В следствие извиканата подпрограма може да променя стойностите на параметрите. Освен това този механизъм позволява ефективно предаване на множества от стойности (напр. структури). Каква е разликата при явно предаване на указател в C?

В някои масови програмни езици съществуват и други средства за предаване на параметри:

Само за четене (read-only) property, напр. за константи в C++, позволява по-ефективно предаване чрез референция.

Параметри по подразбиране (default parameters), което позволява извикваната подпрограма да определя какви параметри да използва, когато при извикването не са зададени параметри или са зададени по-малко на брой параметри.

Позиционно зависими параметри (position-independent parameters), напр. в Ada и LISP, позволява параметрите да бъдат свързвани с имена вместо да бъдат изброявани в строго определен ред.

Управление на паметта

Обикновено програмата използва памет от операционната система, която разделя паметта на страници. Страницата е област от паметта с фиксиран размер, напр. 4-8 KB или повече. Програмата разделя тази памет по собствен начин.

```
a = malloc(12); // Резервиране на 12 байта
```

```
a = b;          // Загуба
```

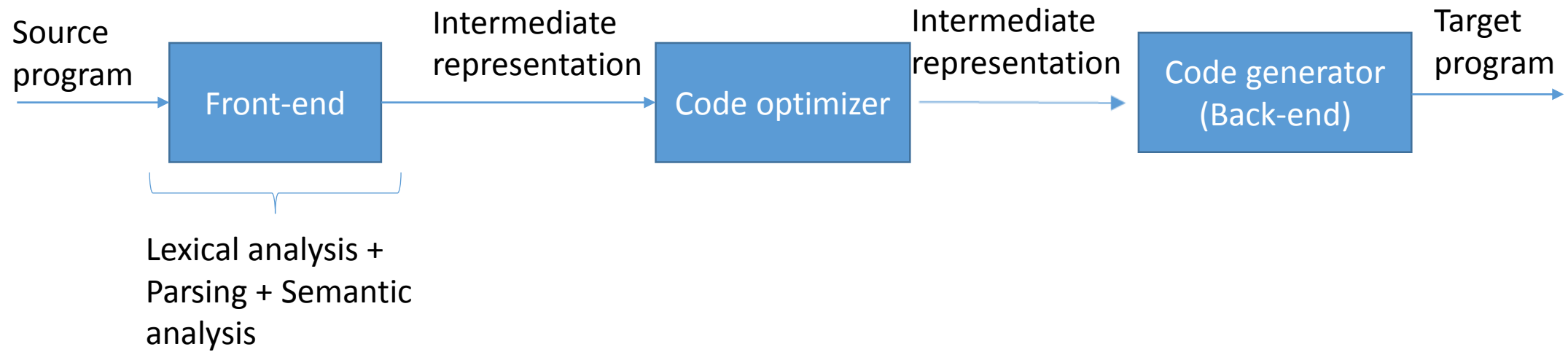
Garbage collection

Някои езици като ML и Java използват т.нар. *събиране на боклука* (*garbage collection*), при което програмистът е освободен от задължението да освобождава динамична памет.

Изпълнителната среда открива, че дадена област от паметта не може да бъде адресирана от програмата, при което областта се освобождава автоматично. Съществуват няколко метода за реализация на garbage collection. Двата най-често използвани методи са *reference counting* и *mark and sweep*.

Автоматичното управление на паметта е много удобно, но понякога причинява проблеми.

Генериране на код



Основни цели на генератора

- Избор на инструкции (instruction selection)
- Заемане на регистри (register allocation and assignment)
- Подреждане на инструкции (instruction ordering)

Цел 1: избор на инструкции

- Какви инструкции да бъдат избрани в съответствие със структурата на целевата машина (target machine)?

Цел 2: заемане на регистри

- Коя стойност в кой регистър да бъде записана (съхранена)?

Цел 3: подреждане на инструкции

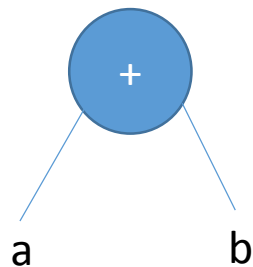
- В какъв ред да бъдат изпълнени генерираните инструкции?

Основни проблеми при проектиране на генератор на код

- Коректност
- Лесна реализация, коректност и поддръжка

Вход на генератора

- Междинна форма (intermediate representation) – синтактично дърво
- Пример: синтактично дърво за израз **a+b**



Исходна програма – целева архитектура

- RISC (reduced instruction set computer)
 - Множество регистри
 - Триадресни инструкции
 - Просто адресиране
 - Ограничен брой прости инструкции
- CISC (complex instruction set computer)
 - Малко регистри
 - Дваадресни инструкции
 - Различни режими за адресиране
 - Сложни инструкции (с променлива дължина)

Исходна програма – целева архитектура

- Стекова машина (stack based)
 - Съхраняване на операнди в стек
 - Изпълнение на операциите с операндите във върха на стека
 - Съхраняване на указател към върха на стека в регистър

Избор на архитектура за генериране на код

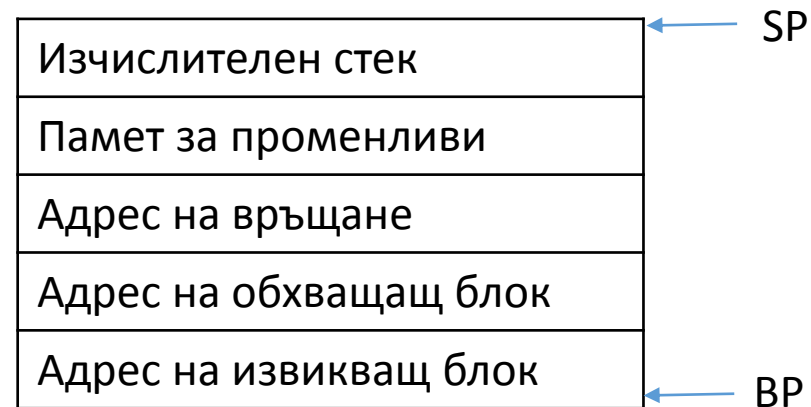
- Универсална стекова машина
 - Операндите се съхраняват в стек
 - Операциите се изпълняват с операндите в стека
- Регистри
 - Програмен брояч – адрес на текущата инструкция
 - Базов указател – начало на фрагмент в стека за текущ блок (главна програма или подпрограма)
 - Стеков указател – адрес на върха на стека (последната заета клетка)

Формат на команда

Код на операцията	Операнд 1	Операнд 2	Адрес на резултата
-------------------	-----------	-----------	--------------------

Стеков фрагмент

- За всеки блок (програма или подпрограма – процедура/функция) се генерира област в паметта (стеков фрагмент) със следната структура



Примерен набор от макрокоманди

- LOAD a – запис на съдържанието на клетка с адрес a във върха на стека
- LDADR a – запис на адреса на a във върха на стека
- STORE a – запис на стойността от върха на стека в клетка с адрес a
- ADD, SUB, MUL, DIV – прочитат двете най-горни клетки в стека, извършват действие, записват резултата в стека

Генериране на команди за аритметични изрази

- Примери

$a+b*c$

$(a+b)*b$

- **Как да се определи приоритет на операциите при сканиране на изречението отляво надясно?**

Премахване на скоби

$(a+b)$

$+(a,b)$

$+ab \rightarrow$ префиксен запис

$ab+ \rightarrow$ постфиксен запис – обратен полски запис (ОПЗ)

- Примери:

$ab+c^* \rightarrow (a+b)^*c$

$a+bc^* \rightarrow ?$

$ab+cd^*+e^*fg+^* \rightarrow ?$

• $abc+^*$

↑

$b+c$
a

← SP

• $abc+^*$

↑

$a*(b+c)$

← SP

Пример за генериране на код

$y := a + b * (c + d)$

ОПЗ $\rightarrow yabcd + * + :=$

y – операнд от адресен тип

a, b, c, d – операнд от стойностен тип

Компилиране в макрокоманди – работна програма

LDADR y

LOAD a

LOAD b

LOAD c

LOAD d

ADD -

MUL -

ADD -

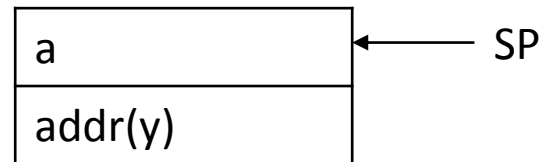
STORE -

Изпълнение на командите в стека

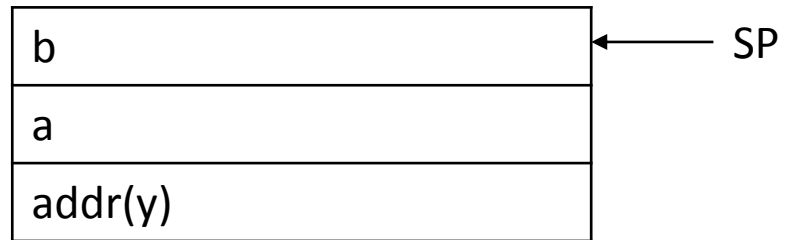
- LDADR y



- LOAD a



- LOAD b



- LOAD c



- LOAD d



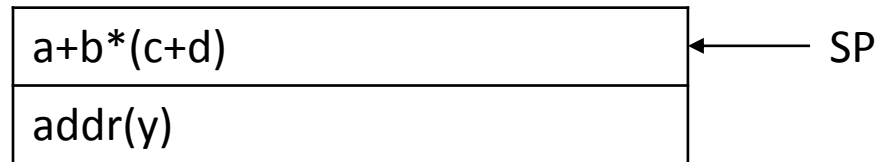
- ADD



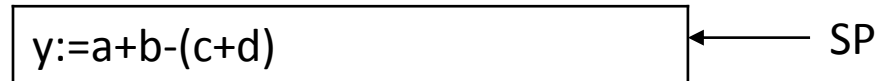
- MUL



- ADD



- STORE



Генериране на код. Тетради.
Генериране на команди за общи
програмни структури, преходи,
условен оператор, оператори за
цикъл

Тетради

- Тетрадите (quadruples) са междинна форма на представяне на генериран код
- Всички оператори могат да бъдат сведени до двуместни (бинарни) и едноместни (унарни)
- Тетрадите представят програмата като последователност от стъпки, в която резултатът от всяка стъпка се съхранява във временна променлива
- В тетрадата за всеки оператор има следните елементи:
 - операцията
 - операндите (един или два)
 - адрес на резултата

(оператор, операнд1, операнд2, резултат)

Примери за тетради

- Пример на тетрада за прост израз:

$B + C \Rightarrow (+, B, C, tmp)$

- По-сложни изрази се преобразуват в множество тетради.

Междинните резултати се съхраняват във временни променливи

$a * (9 + d)$

$\Rightarrow$

$(+, 9, d, tmp1)$

$(*, a, tmp1, tmp2)$

Представяне на общи програмни структури чрез тетради

- Математически и булеви изрази

$a + b \Rightarrow (+, a, b, tmp1)$

$a * (b + c) \Rightarrow (+, b, c, tmp1), (*, a, tmp1, tmp2)$

$a < b \Rightarrow (<, a, b, tmp)$

- Унарни оператори

$-a \Rightarrow (uminus, a, , tmp)$

- Присвояване

$a = a + b \Rightarrow (+, a, b, tmp), (:=, tmp, , a)$

- Декларации

$int\ a, b \Rightarrow -$

$int\ a = 5 \Rightarrow (:=, 5, , a)$

Достъп до елемент на масив и преобразуване на типове

- Достъп до елемент на масив

$a = x[i] \Rightarrow ([]=, x, i, tmp1), (:=, tmp1, , a)$

$x[i] = a \Rightarrow ([]=, x, i, tmp1), (:=, a, , tmp1)$

- Преобразуване на типове

(itof = 'int to float')

- $a = 1; b = a + 0.7$

$\Rightarrow$

$(:=, 1, , a)$

(itof, a, , tmp1)

(+, tmp1, 0.7, tmp2)

$(:=, tmp2, , b)$

Представяне на оператори за преход

- Безусловен преход

(jmp, <jump_address>, ,)

- Условен преход

(<, i, 5, t1) ; условие за преход

(jtrue, L10, t1,)

- Независимо дали преходът е условен или безусловен адресът на прехода е във втория аргумент
- Адресът на прехода представлява цяло число (номер)

Оператор for

- **for (i=0, i<5, i++) do <body> end**

=>

1 (:=, 0, , i) ; инициализация на i

2 (<, i, 5, t1) ; условие за край на цикъла

3 (jfalse, 7, t1, ,)

... <body of loop>

5 (+, i, 1, i) ; увеличаване на i

6 (jmp, 2, ,)

7

Оператор if-else

- **if (a < b) then c = 1 else c = 2**

=>

1 (<, a, b, t1)

2 (jfalse, 5, t1,)

3 (:=, 1, , c)

4 (jmp, 6, ,)

5 (:=, 2, , c)

6

Оператор while

- **while (a < b) do a = a +1**

=>

1 (<, a, b, tmp1)

2 (jfalse, 6, tmp1, ,)

3 (+, a, 1, tmp2)

4 (:=, tmp2, , a)

5 (jmp, 1, ,)

6

Упражнение

- Дадена е следната програма:

```
int a = 2, b = 8, c = 4, d;
```

```
for (j=0; j<=10; j++){
```

```
  a = a * (j* (b/c));
```

```
  d = a * (j* (b/c));
```

```
}
```

- Да се покаже генерираният междинен код във вид на теради